

Curso Bases de Datos Relacionales

Introducción a las Bases de Datos

Introducción a SQL

SQL, o Structured Query Language, es el lenguaje estándar utilizado para interactuar y gestionar bases de datos relacionales. Desde su creación en la década de 1970 por IBM, SQL se ha consolidado como una herramienta fundamental para la manipulación y permanencia de datos en el ámbito de las tecnologías de la información. Su adopción se ha extendido globalmente, convirtiéndose en el pilar central de muchos sistemas de gestión de bases de datos (SGBD) como PostgreSQL, MySQL, Oracle, y Microsoft SQL Server.

SQL es un lenguaje y lógica de gestión de bases de datos en formato tabular que permite a los usuarios definir, manipular y controlar el acceso a los datos de una manera estructurada y eficiente. Con SQL, es posible realizar una amplia gama de operaciones: desde la creación de estructuras de datos complejas hasta la inserción, actualización, eliminación y consulta de registros específicos. Su sintaxis clara y declarativa facilita tanto tareas simples como la recuperación de datos, como operaciones más complejas que involucran múltiples tablas y condiciones que buscan adaptarse a las necesidades y preguntas de negocio, incluyendo el establecimiento de roles de usuarios que poseen diferentes niveles acceso o capacidad de edición sobre las bases de datos.

Una de las características más destacadas de SQL es su capacidad para trabajar con grandes volúmenes de datos de manera rápida y precisa. Esta versatilidad se debe, en parte, a su estructura basada en sentencias que permiten especificar exactamente qué datos se desean manipular y cómo. Las sentencias de SQL, como SELECT, INSERT, UPDATE, DELETE, entre otras, permiten realizar casi cualquier tipo de operación imaginable en una base de datos. El diseño de SQL sigue principios matemáticos basados en el álgebra relacional que garantizan consistencia y precisión en el manejo de datos. Por esta razón, SQL es fundamental no solo para desarrolladores y administradores de bases de datos, sino también para científicos de datos, analistas de negocio, y cualquier profesional que necesite trabajar con datos de manera eficiente y segura.

Corresponde a un paradigma del manejo de la información tan sólido que a pesar de que su aparición se da en la década de los setenta, su sintaxis y lógica siguen invariantes desde la fecha, siendo aún hoy en día uno de los lenguajes y herramientas más usadas en el mundo de la tecnología, constituyendo una habilidad no solo necesaria sino ineludible para todo profesional en datos. Con su amplio soporte por parte de múltiples plataformas y su capacidad de integración con diversos lenguajes de programación, SQL sigue siendo relevante y se adapta a las necesidades del entorno tecnológico moderno. Aprender SQL es un paso crucial para cualquier persona que desee comprender y utilizar de manera efectiva las bases de datos relacionales en el mundo de hoy.

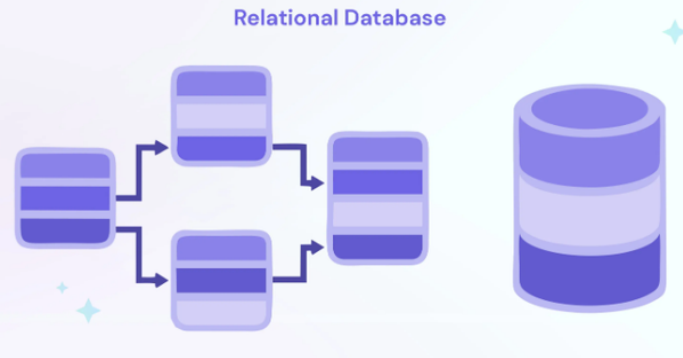


figura 1: Ilustración de una base de datos relacional, la cual está constituida por la colección interconectada de tablas de datos.

SQL son las siglas de **Structured Query Language** (Lenguaje de Consulta Estructurada), es un lenguaje de programación diseñado específicamente para gestionar y manipular datos en bases de datos relacionales. Recibe su nombre debido a su capacidad para formular consultas de manera estructurada, es decir, con una sintaxis clara y precisa que permite definir, buscar, insertar, modificar y eliminar datos de manera eficiente. El paradigma de SQL es declarativo: en lugar de especificar cómo realizar una operación, el usuario indica *qué* quiere lograr, y el sistema se encarga de determinar la mejor manera de ejecutar la instrucción. Esta característica lo diferencia de otros lenguajes de programación imperativos donde se debe detallar cada paso del proceso.

Lo que caracteriza a SQL es su simplicidad y versatilidad; es capaz de manejar grandes volúmenes de datos con pocas líneas de código, trabajar con múltiples tablas a través de **joins** o uniones, y aplicar funciones agregadas para obtener información estadística o resumida. Además, su estandarización asegura que pueda ser utilizado en casi todos los sistemas de gestión de bases de datos, lo que lo convierte en una herramienta universal para la gestión y análisis de datos independientemente del proveedor de software.

Los proveedores de manejadores de bases de datos relacionales más célebres son:













Consistencia ACID en Bases de Datos: Un Pilar Fundamental de la Integridad de los Datos

La consistencia ACID es un conjunto de propiedades esenciales que garantizan la integridad, fiabilidad y solidez de las transacciones en bases de datos. El acrónimo ACID representa los términos *Atomicidad, Consistencia, Aislamiento y Durabilidad*, que son principios clave en el diseño de sistemas de bases de datos relacionales.

Estas propiedades aseguran que las operaciones de las bases de datos se ejecuten de manera segura, consistente y predecible, evitando corrupción de datos o problemas de integridad. Cada una de estas propiedades juega un papel vital para garantizar que las transacciones en la base de datos se comporten de manera fiable, incluso en situaciones adversas, como fallos de hardware, errores de software o cortes de energía.

1. **Atomicidad (Atomicity):** La atomicidad asegura que una transacción se ejecute de manera "todo o nada". Esto significa que una transacción es una unidad indivisible: si alguna parte de la transacción falla, la totalidad de la transacción debe fallar y no se debe realizar ningún cambio en la base de datos. En otras palabras, una transacción es atómica si se completan todas sus operaciones o si ninguna de ellas se lleva a cabo. Imaginemos una transacción bancaria que implica transferir dinero de la cuenta A a la cuenta B. La transacción debe restar una cantidad X de la cuenta A y añadirla a la cuenta B. Si se realiza el débito de la cuenta A pero el crédito a la cuenta B falla, el sistema debe revertir la operación para mantener la coherencia de los datos, asegurando que ninguna de las operaciones tenga efecto.
2. **Consistencia (Consistency):** La consistencia garantiza que una transacción llevará a la base de datos de un estado válido a otro estado igualmente válido. Esto significa que una

transacción debe respetar todas las reglas y restricciones de integridad definidas en la base de datos, como restricciones de claves primarias, claves foráneas, unicidad, y reglas de negocio específicas. La consistencia asegura que, después de que una transacción se complete con éxito, todos los datos de la base de datos estarán en un estado consistente. Si una transacción termina con errores, cualquier cambio realizado durante la transacción debe deshacerse para restaurar la base de datos a su estado previo. Por ejemplo, si una base de datos contiene una restricción que establece que el saldo de una cuenta no puede ser negativo, cualquier transacción que viole esta regla será rechazada, y el estado de la base de datos se mantendrá consistente conforme a las reglas definidas.

3. **Aislamiento (Isolation):** El aislamiento asegura que las operaciones de una transacción sean invisibles para otras transacciones hasta que la transacción en curso se haya completado con éxito. Esto significa que el resultado de una transacción no debe ser visible a otras transacciones en curso hasta que esta se haya confirmado (o deshecho en caso de fallo). El aislamiento previene las anomalías como las lecturas no repetibles, las lecturas sucias, y las lecturas fantasma, que pueden ocurrir cuando varias transacciones se ejecutan simultáneamente sin las garantías adecuadas de aislamiento. Los sistemas de bases de datos suelen ofrecer diferentes niveles de aislamiento, como "lectura no confirmada" (read uncommitted), "lectura confirmada" (read committed), "repetible" (repeatable read) y "serializable", siendo este último el nivel de aislamiento más estricto. Un ejemplo de la propiedad de aislamiento corresponde a suponer dos transacciones que intentan actualizar el mismo registro de cliente al mismo tiempo: una intenta cambiar el nombre del cliente y otra actualiza su dirección. Si el aislamiento no se mantiene, una transacción podría leer un estado intermedio de los datos mientras la otra transacción está en curso, lo que llevaría a inconsistencias. Un nivel de aislamiento adecuado asegurará que cada transacción se ejecute de manera independiente de las otras.
4. **Durabilidad (Durability):** La durabilidad garantiza que, una vez que una transacción ha sido confirmada (committed), los cambios realizados a la base de datos son permanentes, incluso si ocurre un fallo del sistema o una caída de energía inmediatamente después. Esto se logra mediante el uso de mecanismos de almacenamiento persistente, como logs de transacciones o técnicas de replicación. Para mantener la durabilidad, los sistemas de bases de datos escriben cambios a un log o registro de transacciones en el disco antes de confirmar la transacción. En caso de un fallo del sistema, el sistema puede recuperarse utilizando este log para restaurar todas las transacciones confirmadas. Por ejemplo, consideremos que se realiza una transacción de compra en un sistema de comercio electrónico que implica registrar la compra en la base de datos. Una vez que esta transacción se confirma, la información de la compra queda registrada permanentemente, incluso si el servidor se apaga repentinamente. Al reiniciar, el sistema puede utilizar los logs de transacciones para garantizar que la compra no se pierda.

En entornos donde las transacciones son críticas (como bancos, e-commerce, y sistemas de reservas), las propiedades ACID garantizan que las operaciones complejas puedan ejecutarse de manera segura, minimizando el riesgo de pérdida de datos y garantizando la confiabilidad del sistema.

Historia y Evolución de SQL

SQL nació a principios de la década de 1970 como respuesta a la necesidad creciente de manejar grandes cantidades de información de manera eficiente y estructurada. La génesis de SQL se remonta al trabajo pionero de **Edgar F. Codd**, un científico de IBM que, en 1970, publicó un influyente artículo titulado *"A Relational Model of Data for Large Shared Data Banks"*. En este artículo, Codd presentó por primera vez el concepto de bases de datos relacionales, basadas en el uso de tablas para almacenar información de manera estructurada siendo cada columna de la tabla algún tipo de propiedad mientras que las filas correspondían a los registros. Esta idea revolucionaria sentó las bases para el desarrollo de SQL, ya que estableció los principios matemáticos del álgebra relacional, que hoy son fundamentales para el manejo de datos.



figura 2: Instalaciones de IBM y Oracle.

En 1973, un grupo de investigadores de IBM liderados por Donald D. Chamberlin y Raymond F. Boyce tomó las ideas de Codd y desarrolló un lenguaje de consulta llamado **SEQUEL** (Structured English Query Language) para interactuar con su prototipo de base de datos relacional llamado **System R**. El propósito de SEQUEL era permitir a los usuarios recuperar (consultar) y manipular datos utilizando un lenguaje más cercano al inglés que al código de máquina tradicional. Sin embargo, debido a problemas de marca registrada con el nombre "SEQUEL", el lenguaje fue renombrado simplemente como SQL.

Durante los siguientes años setenta e inicios de los ochenta, IBM continuó perfeccionando SQL, y en 1979, una pequeña empresa llamada Relational Software Inc., que más tarde se convertiría en **Oracle Corporation**, lanzó su propia implementación de SQL para su base de datos relacional comercial. Oracle fue el primer SGBD (Sistema de Gestión de Bases de Datos) en comercializar con éxito una solución basada en SQL, marcando el comienzo de su popularidad en la industria. Poco después, en 1981, IBM introdujo SQL/DS, su propio sistema de base de datos relacional, seguido en 1983 por DB2, ambas soluciones cimentando el estatus de SQL como el estándar para las bases de datos relacionales.

La estandarización de SQL llegó en 1986 cuando el Instituto Nacional Estadounidense de Estándares (ANSI) aprobó la primera versión estándar de SQL, que más tarde fue ratificada por la Organización Internacional de Normalización (ISO). Este estándar ayudó a consolidar el uso de SQL en diferentes sistemas de gestión de bases de datos, lo que facilitó la interoperabilidad entre ellos y promovió una adopción aún mayor. Las versiones posteriores

del estándar, como SQL-89, SQL-92, SQL:1999 (introduciendo características como triggers y procedimientos almacenados), SQL:2003 (con soporte para XML), SQL:2011 (con funciones temporales) y SQL:2016 (añadiendo soporte para JSON), reflejan la continua evolución del lenguaje para satisfacer las necesidades de la industria.

Durante los años 90 y 2000, SQL experimentó una adopción masiva gracias a la expansión de Internet y la necesidad de manejar datos a gran escala. Empresas como Microsoft con SQL Server, Oracle con sus productos, y la comunidad de software libre con bases de datos como MySQL y PostgreSQL adoptaron SQL como su lenguaje principal, fortaleciendo su posición en el ecosistema tecnológico.

Hoy en día, SQL sigue siendo el estándar de facto para la gestión de datos relacionales, utilizado en una amplia variedad de aplicaciones, desde pequeños sistemas locales hasta gigantescos sistemas distribuidos que manejan petabytes de información. Su simplicidad, robustez y versatilidad lo han mantenido relevante, adaptándose a nuevas tecnologías como la computación en la nube y el Big Data, evolucionando para incorporar nuevas características que lo hacen adecuado para una amplia gama de casos de uso modernos.

La historia de SQL es, en esencia, la historia de cómo la gestión de datos ha evolucionado desde un desafío técnico para científicos de la computación hasta convertirse en un pilar fundamental de la economía digital actual. Desde sus humildes comienzos en los laboratorios de IBM hasta su adopción global como el lenguaje universal para las bases de datos, SQL ha sido y seguirá siendo ser una herramienta indispensable para todos aquellos que trabajan con datos.

Tipos de bases de datos: de relacionales a no relacionales

Las bases de datos han evolucionado significativamente desde sus inicios, adaptándose a las necesidades cambiantes de las organizaciones y a los avances tecnológicos. A lo largo de las décadas, diferentes tipos de bases de datos han surgido para satisfacer las diversas demandas del almacenamiento, procesamiento y acceso a la información. Esta evolución ha sido impulsada tanto por la creciente complejidad de los datos como por la necesidad de manejar volúmenes masivos de información de manera eficiente.

Algunas ventajas de las bases de datos relacionales son:

1. **Consistencia y Precisión:** A través del uso de restricciones, claves primarias y foráneas, y transacciones ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), las bases de datos relacionales garantizan la integridad y la coherencia de los datos.
2. **Flexibilidad de Consultas:** SQL permite realizar consultas complejas y unir múltiples tablas para obtener información detallada.
3. **Estándares de la Industria:** Las bases de datos relacionales son ampliamente aceptadas y estandarizadas, con un sólido respaldo de proveedores y una comunidad de usuarios extensa.

No obstante, las bases de datos relacionales poseen limitaciones como:

1. **Escalabilidad Limitada:** Las bases de datos relacionales están diseñadas para escalar verticalmente (aumentando la capacidad de un solo servidor), lo que puede resultar costoso y no siempre viable. No se adaptan bien a la escalabilidad horizontal, que implica añadir más servidores para gestionar una carga de trabajo creciente.
2. **Rigidez del Esquema:** El modelo relacional requiere un esquema de datos (modelo de relacionamiento de datos) predefinido y rígido. Esto hace que sea difícil y costoso adaptarse a cambios en la estructura de los datos, lo que puede ser una limitación importante en aplicaciones modernas donde la agilidad es crucial.
3. **Desempeño en Datos No Estructurados:** Las bases de datos relacionales no manejan eficientemente datos no estructurados o semiestructurados como documentos, multimedia, o datos de redes sociales, que no encajan bien en el modelo tabular tradicional.
4. **Complejidad en Big Data:** En aplicaciones de Big Dat, donde se manejan volúmenes enormes de datos a alta velocidad (conocido como las tres Vs: Volumen, Velocidad, Variedad), las bases de datos relacionales pueden presentar problemas de rendimiento.

Debido a estas limitaciones, han surgido nuevas tecnologías de bases de datos, conocidas como bases de datos no relacionales, diseñadas para abordar estos desafíos específicos.

Las bases de datos no relacionales, comúnmente conocidas como **NoSQL (Not Only SQL)**, han emergido como una solución alternativa para los problemas que enfrentan las bases de datos relacionales en el manejo de datos grandes y diversos. Estas bases de datos están diseñadas para proporcionar flexibilidad, escalabilidad horizontal y eficiencia en el manejo de datos no estructurados o semiestructurados.

A diferencia de las bases de datos relacionales, las NoSQL no siguen un modelo tabular estricto. En su lugar, ofrecen diferentes enfoques para almacenar y recuperar datos, cada uno optimizado para casos de uso específicos.

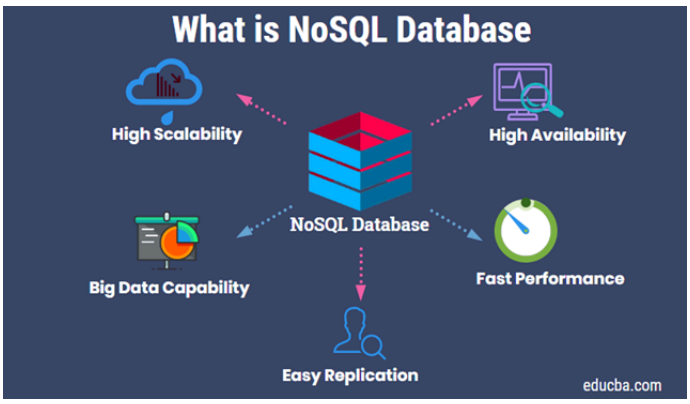


figura 3: Propiedades de una base de datos NoSQL.

Algunas de las propiedades más relevantes de las bases de datos SQL y NoSQL son:

Propiedad	Bases de datos SQL	Bases de datos NoSQL
Modelo de Datos	Modelo de data relacional tabular.	Modelo flexible, adaptable a diveras fuentes y tipos de datos.
Esquema	Esquema estructurado predefinido antes de la inserción de data.	Esquema dinámico sin restricciones, no definido antes de la inserción de data.
Escalabilidad	Escalabilidad vertical.	Escalabilidad horizontal.
Lenguaje de consulta	SQL	Diversos tipos de lenguaje de consulta, único para cada tipo de base de datos. Posiblemente dependiente del software empleado.
ACID Compliance	Compatibilidad ACID(Atomicity, Consistency, Isolation, Durability)	Puede o no tener compatibilidad ACID dependiendo de la base de datos NoSQL específica.
Flexibilidad de escalabilidad	Escalabilidad limitada debido a su estructura relacional.	Alta escalabilidad, idal para sistemas distribuidos.
Integridad de los datos	Alta consistencia de los datos, reforzada por restricciones de integridad y el modelo relacional.	Puede sacrificar la consistencia de los datos a favor de la disponibilidad de los datos y su particionamiento efectivo.
Flexibilidad	Menos flexible para trabajar datos no estructurados, dificultades en la consulta de este tipo de datos.	Alta flexibilidad para lidiar con datos no estructurados, semiestructurados y de estructura variable.
Casos de uso tradicionales	Uso tradicional en organizaciones que basen sus operaciones en datos tabulares.	BigData, aplicaciones de tiempo real en la web, sistemas altamente escalables.

Las bases de datos NoSQL nunca siguen el modelo relacional, por lo tanto, nunca proporcionan tablas con registros planos de columnas fijas y no requieren mapeo objeto-relacional ni normalización de datos. Como no poseen un esquema explícito, se dice que son

"libres de esquema" o con *"esquema implícito"* pero carecen de un lenguaje de consulta estándar como en el caso de las bases de datos SQL. Dado que las bases de datos NoSQL están pensadas principalmente para usos de alto rendimiento, cada proveedor de DBMS NoSQL está preparado para ofrecer una gestión de datos distribuida. Se dice entonces que las bases de datos NoSQL:

- Ofrecen capacidades horizontales de autoescalado y conmutación por error.
- A menudo, el concepto ACID puede escarificarse para mejorar la escalabilidad y el rendimiento.
- En la mayoría de los casos no hay replicación síncrona entre nodos distribuidos.
- Arquitectura de Nada Comúnmente Compartida.

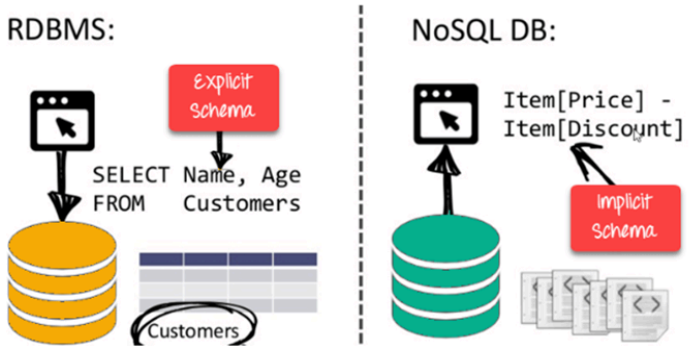


figura 4: Comparación entre una sentencia SQL con esquema explícito y una sentencia NoSQL con esquema implícito.

En general, si se compara una base de datos SQL y NoSQL se puede establecer la siguiente relación o paralelismo entre los elementos que las componen:

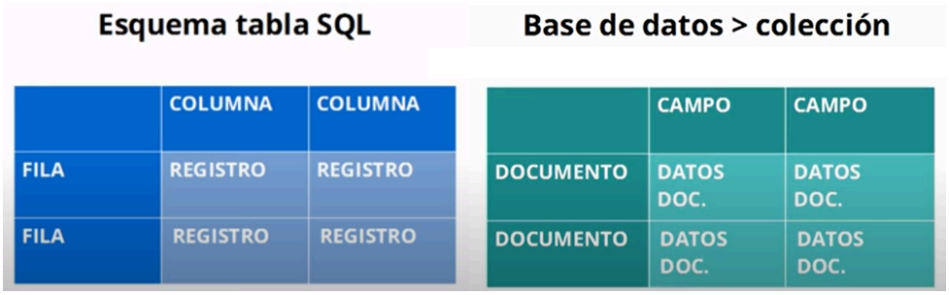


figura 5: Comparación de elementos constitutivos de un esquema SQL y NoSQL.

SQL	NoSQL
Database	Collection
Row	Documents
Column	Fields
Registers (data cells)	Data of the Document

Las bases de datos NoSQL se clasifican principalmente en cuatro tipos: *Par clave-valor, Orientadas a columnas, Basadas en grafos y Orientadas a documentos.* Cada categoría tiene sus propios atributos y limitaciones. Ninguna de las anteriores es mejor para resolver todos los problemas, de esta manera los usuarios deben seleccionar la base de datos en función de las necesidades de su producto.

Dentro de las bases de datos NoSQL existe algo llamado el teorema **CAP**, que afirma que es imposible que un almacén de datos distribuido ofrezca más de dos de tres de las siguientes garantías:

1. **Consistencia:** Los datos deben permanecer consistentes incluso después de la ejecución de una operación. Esto significa que una vez escritos los datos, cualquier futura petición de lectura debe contener esos datos. Por ejemplo, después de actualizar el estado del pedido, todos los clientes deberían poder ver los mismos datos.
2. **Disponibilidad:** Las bases de datos deben estar siempre disponibles y responder. No debe tener ningún tiempo de inactividad.
3. **Tolerancia a la partición:** Tolerancia a la partición significa que el sistema debe seguir funcionando aunque la comunicación entre los servidores no sea estable. Por ejemplo, los servidores pueden dividirse en varios grupos que no se comuniquen entre sí. En este caso, si una parte de la base de datos no está disponible, las demás partes no se verán afectadas.

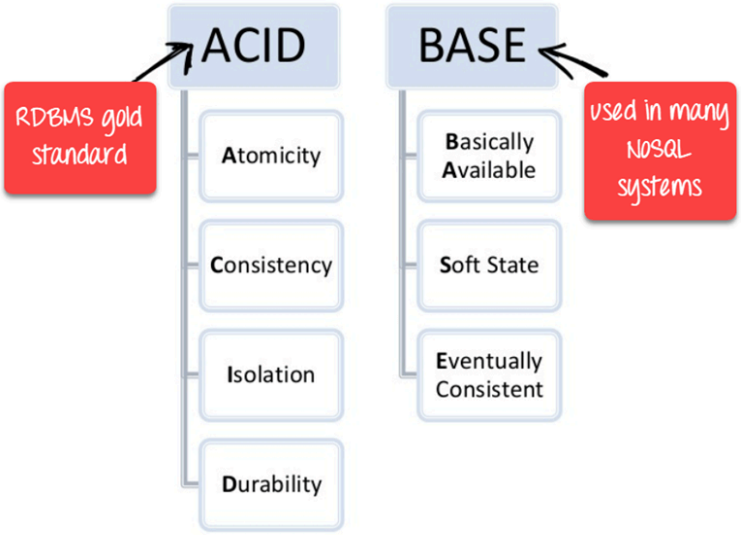


figura 6: Comparación entre las compatibilidad ACID y BASE característicos de las bases de datos SQL y NoSQL respectivamente.

Vimos anteriormente que las bases de datos SQL se rigen por una total consistencia y un paradigma ACID, por el contrario, las bases de datos NoSQL se caracterizan por la **consistencia Eventual** lo cual significa tener copias de datos en múltiples máquinas para obtener alta disponibilidad y escalabilidad que eventualmente tendrán la misma información actualizada a lo largo de todas las copias distribuidas. Por lo tanto, los cambios realizados en cualquier elemento de datos en una máquina tienen que propagarse a otras réplicas.

La replicación de datos puede no ser instantánea, ya que algunas copias se actualizarán inmediatamente y otras a su debido tiempo. Estas copias pueden ser mutuas, pero con el tiempo se vuelven consistentes. De ahí el nombre de consistencia eventual. Por otra parte el paradigma **BASE (Basically Available, Soft state, Eventual consistency)** característico de las bases de datos NoSQL implica:

1. **Básicamente Disponible:** Significa que la base de datos está disponible todo el tiempo basándose en el teorema CAP (relacionado principalmente con la disponibilidad).
2. **Soft state:** Significa que incluso sin una entrada, el estado del sistema puede cambiar (relacionado con la coherencia de los datos).
3. **Consistencia eventual:** Significa que el sistema se volverá consistente con el tiempo (tolerancia a la partición y actualización).

Los tipos de bases de datos no relacionales, dependen del caso de uso y el tipo de aplicación de interés. En general, los tipos de bases de datos no relacionales más comunes son:

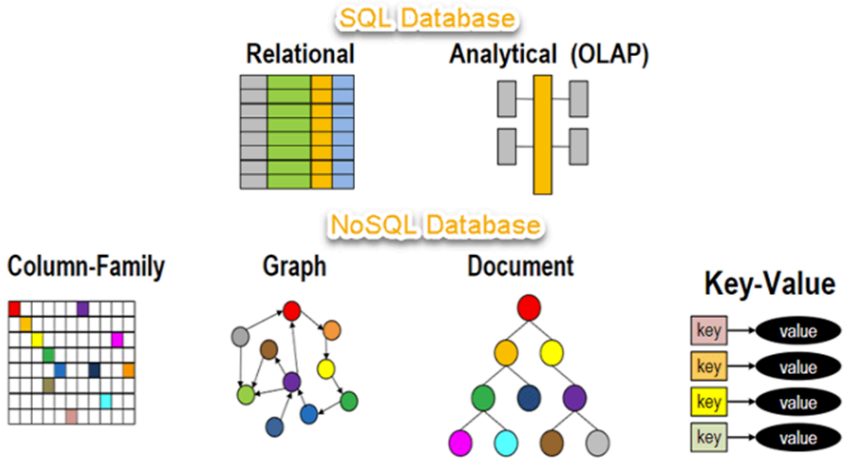


figura 7: Tipos de bases de datos más comunes.

Bases de Datos Clave-Valor:

Los datos se almacenan en pares clave/valor, y están diseñadas para manejar grandes cantidades de datos y cargas pesadas. Las bases de datos de almacenamiento de pares clave-valor similar a un diccionario, almacenan los datos como una tabla hash en la que cada clave es única, y el valor puede ser una cadena JSON, BLOB (Binary Large Objects). La llave es un identificador único que se utiliza para buscar su valor correspondiente. Algúnos ejemplos de manejadores de bases de datos Clave-valor son **Redis, y Amazon DynamoDB.**

Algunas de sus ventajas son su escalabilidad horizontal extrema, simplicidad, y alto rendimiento en operaciones de lectura y escritura. Son usadas en el almacenamiento de sesiones, caché de aplicaciones, almacenamiento de configuraciones, almacenamiento de datos textuales, entre otras.

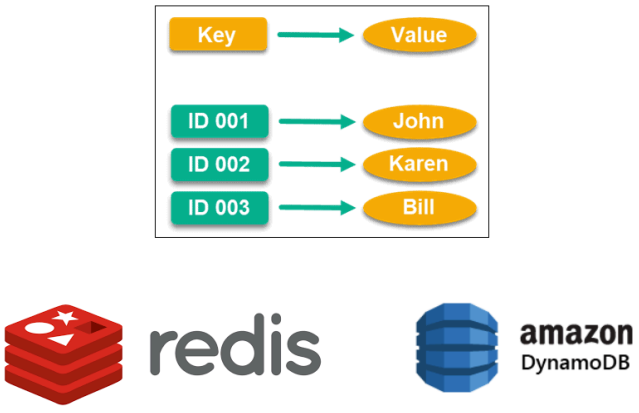


figura 8: Ilustración de una base de datos llave-valor y logo de Redis y Amazon DynamoDB.

Bases de Datos Columnares o de Columna Ancha

Organizan los datos en filas y columnas, pero a diferencia de las bases de datos relacionales, permiten un esquema dinámico donde cada fila puede tener un número diferente de

columnas. Los datos se almacenan por columnas en lugar de por filas. Algunos ejemplos son **Apache Cassandra** y **HBase**.

Algunas de sus ventajas son la alta eficiencia en operaciones de escritura y lectura a gran escala, buena compresión de datos, y capacidad para manejar grandes volúmenes de datos distribuidos. Usadas en el análisis de series temporales, almacenamiento de datos en tiempo real, y aplicaciones de IoT (Internet de las Cosas).

	Datos personales		Datos profesionales	
Clave	Nombre	Edad	Puesto	Salario
1	Nombre1	25	RRHH	50000
2	Nombre2	33	formáti	50000
3	Nombre3	60	Finanzas	50000
4	Nombre4	28	Ventas	50000
5	Nombre5	40	Logística	50000





figura 9: Ilustración de una base de datos columnar y logo de Apache Cassandra y Apache HBase.

Bases de Datos Documentales:

Almacenan datos en forma de documentos, generalmente en formato JSON o BSON. Cada documento puede tener un esquema flexible, permitiendo diferentes estructuras de datos dentro de la misma colección. Algunos de sus ejemplos son **MongoDB, CouchDB, Amazon SimpleDB, Riak** y **Lotus Notes.**

Dentro de sus ventajas encontramos su flexibilidad del esquema, facilidad para manejar datos semi-estructurados o no estructurados, y capacidades de indexación para consultas rápidas. Son usadas en aplicaciones web y móviles, sistemas de gestión de contenidos, análisis de datos en tiempo real.

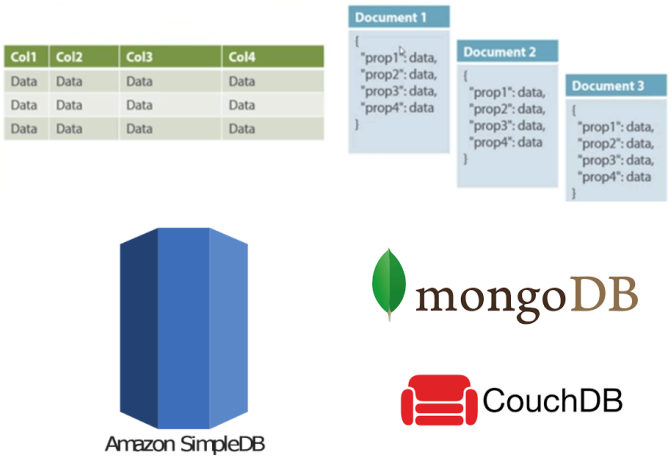


figura 10: Ilustración de una base de datos documental en su forma SQL (panel superior izquierdo) y su forma documental NoSQL (panel superior derecho). Logos de MongoDB, CouchDB y Amazon SimpleDB.

Bases de Datos Basadas en Grafos

Están diseñadas para gestionar relaciones complejas entre datos, utilizando nodos, aristas y propiedades para representar y almacenar información. Son especialmente adecuadas para manejar datos conectados. Las bases de datos gráficas más célebres son **Neo4j** y **Amazon Neptune.** Poseen una alta eficiencia en la gestión de relaciones complejas, rapidez en la ejecución de consultas sobre grafos densos, y facilidad para modelar redes sociales, sistemas de recomendación, y grafos de conocimiento. Comúnmente usadas en el manejo de redes sociales, motores de recomendación, detección de fraudes, y análisis de redes.

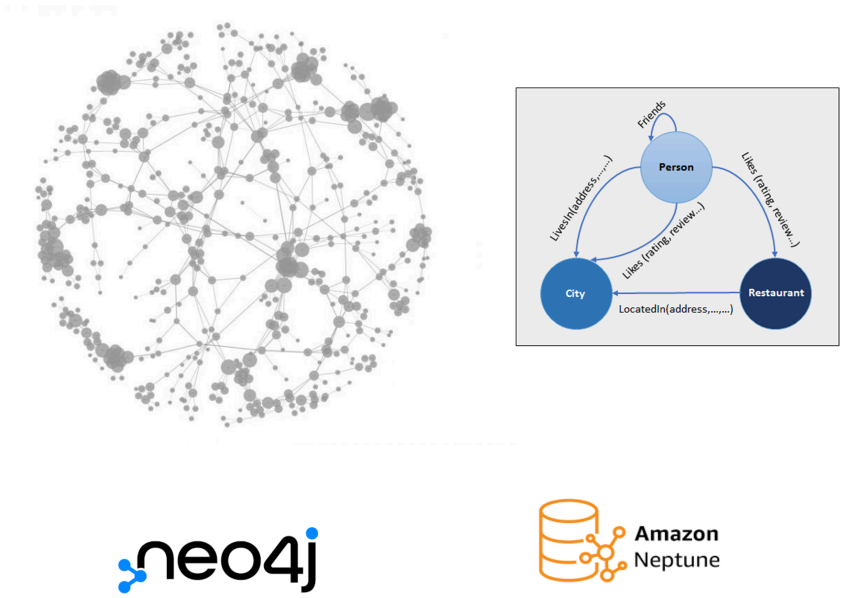


figura 11: Ilustración de una base de datos basada en grafos y logos de Neo4J y Amazon Neptune.

A medida que las aplicaciones modernas han evolucionado para manejar un volumen, velocidad y variedad de datos sin precedentes, las bases de datos relacionales, aunque

robustas y confiables, no siempre son la mejor solución. Las bases de datos no relacionales han surgido como alternativas necesarias para satisfacer las nuevas demandas de escalabilidad, flexibilidad y rendimiento. Al optar por el tipo de base de datos adecuado según las necesidades específicas del caso de uso, las organizaciones pueden asegurar una gestión eficiente de sus datos en el entorno tecnológico actual.

Fundamentos de SQL

Entidades, relaciones y conceptos básicos

Las bases de datos relacionales son estructuras organizadas que permiten almacenar y gestionar datos de manera eficiente mediante el uso de tablas, donde los datos se agrupan de forma lógica. A continuación, se presentan los conceptos esenciales:

Entidades y sus Tipos

Una entidad representa cualquier objeto o concepto del mundo real sobre el cual se necesita almacenar información en una base de datos. Por ejemplo, una entidad puede ser un cliente, un producto o una orden. Las entidades se caracterizan por tener atributos específicos que describen sus propiedades, como el nombre y la dirección de un cliente o el precio de un producto. Dependiendo de su relación con otras entidades, pueden clasificarse como entidades **fuertes** (existentes independientemente) o **débiles** (cuya existencia depende de otra entidad).

- **Entidades fuertes:** Entidades que por definición, existen por sí mismas. Por ejemplo, en un sistema de gestión de una tienda, un cliente puede existir independientemente de otros elementos del sistema. Un cliente tendría atributos como ID de cliente, nombre, dirección, y correo electrónico. La clave primaria aquí es el *ID de cliente*, que identifica de forma única a cada cliente.
- **Entidades débiles:** Entidades que no pueden existir por sí mismas y dependen de otra entidad para existir. Por ejemplo, en un sistema de gestión de pedidos, la entidad "Detalle de Orden" no puede existir sin una entidad "Orden" asociada. Cada detalle de orden está vinculado a una orden específica y puede tener atributos como cantidad, precio unitario y producto, pero depende del número de orden (entidad fuerte) para existir. El número de orden funciona como una clave foránea que asegura su existencia.
- **Entidades débiles por identidad:** No se pueden distinguir entre sí a menos que se les asocie una clave de una entidad fuerte. Por ejemplo, en el contexto de un hotel, una habitación puede ser una entidad débil por identidad. Sin una relación con la entidad fuerte "Hotel", una habitación solo identificada por el número "101" no tiene significado

único, ya que diferentes hoteles pueden tener una habitación con el mismo número. Al asociarse con la clave primaria del "Hotel" (como el nombre o el ID del hotel), la habitación se distingue, por ejemplo, "Habitación 101 del Hotel A".

- **Entidades débiles por existencia:** Entidades débiles a las que deben asignársele su propio identificador. Por ejemplo, en una empresa, cada empleado (entidad fuerte) puede tener un seguro de vida asociado. La entidad "Seguro de Vida" es débil por existencia porque depende del empleado para su existencia. Sin embargo, cada seguro de vida debe tener su propio identificador único (como el número de póliza) además de la clave del empleado para distinguir diferentes seguros de vida, especialmente si un empleado tiene múltiples pólizas.

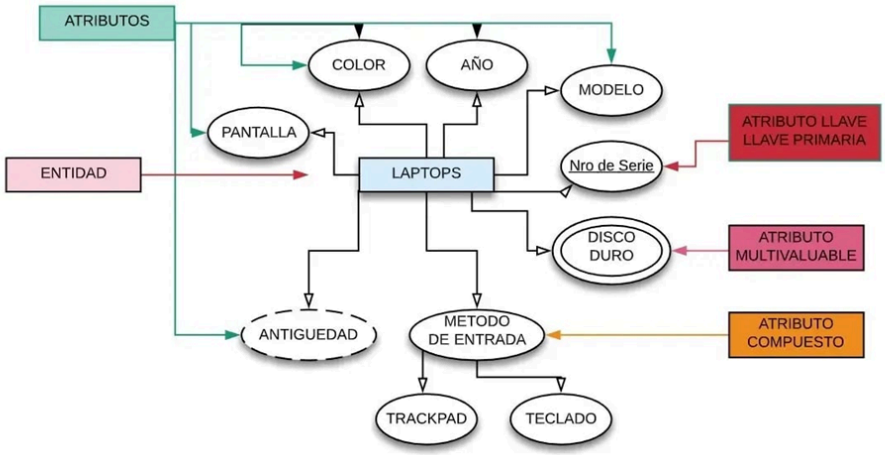


figura 12: Diagrama ER que define una entidad como la colección interconectada de sus atributos.

Relaciones

Las relaciones describen cómo interactúan o se conectan diferentes entidades dentro de una base de datos. Estas relaciones se establecen mediante claves primarias y foráneas. Por ejemplo, un cliente puede tener una relación de "hacer" con una orden de compra. Las relaciones pueden ser de cero a uno (0:1), cero a muchos (0,N), uno a uno (1:1), uno a muchos (1:N), o muchos a muchos (M:N), dependiendo del contexto de los datos y las reglas de negocio.

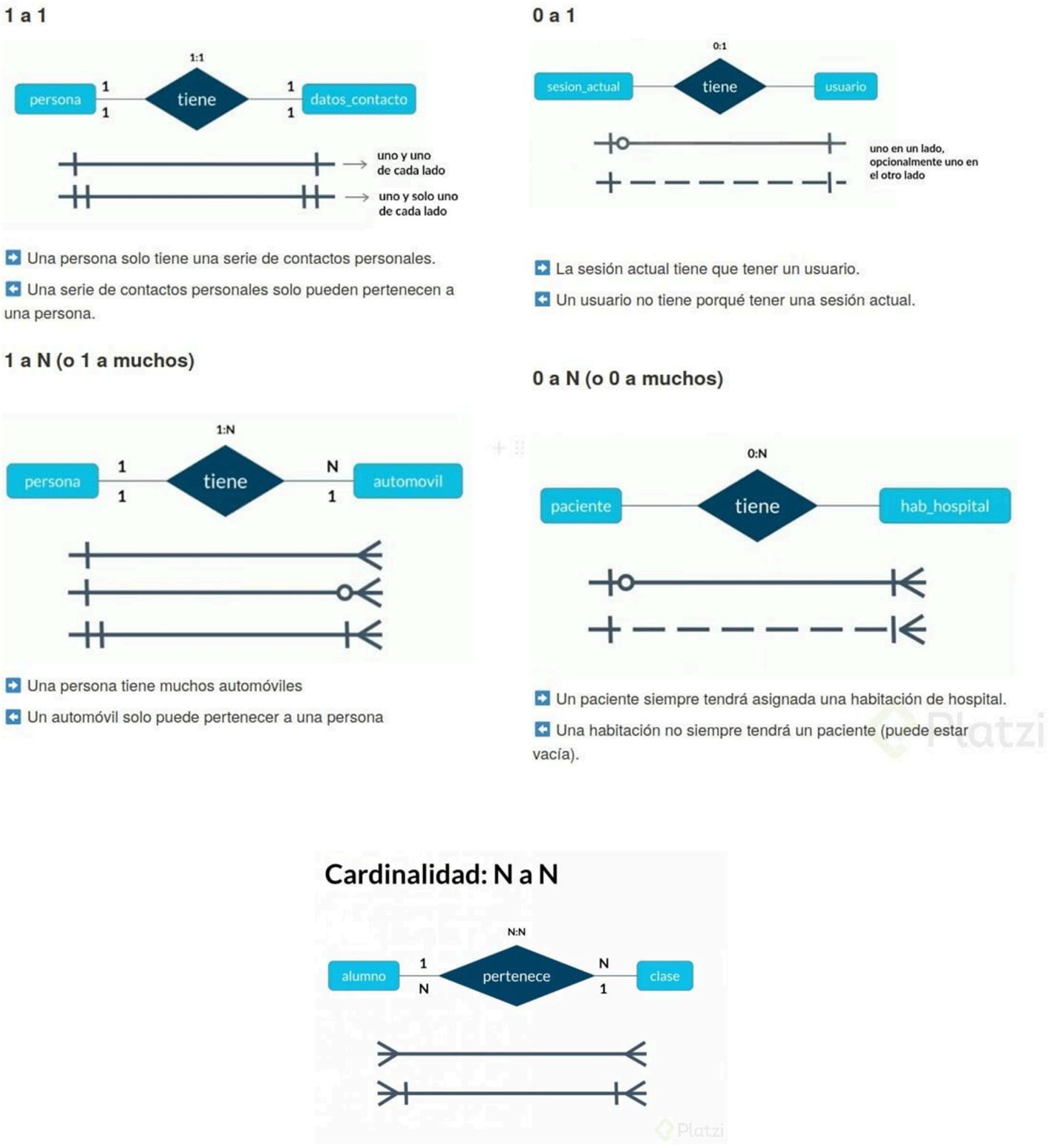


figura 13: Tipos de relaciones en bases de datos SQL.

Los tipos más comunes de relaciones son:

1. **Uno a uno (1:1)**: Entre una tabla A y B, existe una única relación entre sus elementos, es decir, cada elemento de A con vínculo hacia B (o viceversa) solo puede conectarse a un único elemento en B.
2. **Cero a uno (0:1):** Entre una tabla A y B existe una única relación entre sus elementos que puede ser opcional, la existencia del vínculo puede o no presentarse entre los registros de las tablas.

3. **Uno a muchos (1:N):** Entre una tabla A y B, existe una relación desde un elemento de A hacia múltiples elementos B, es decir, un elemento de A es asociado a varios elementos en B.
4. **Cero a muchos (0:N):** Entre una tabla A y B, existe una relación opcional entre un elemento de A y múltiples elementos de B.
5. **Muchos a muchos (M:N):** Entre una tabla A y B, existen múltiples relaciones entre los elementos de ambas tablas, corresponde la relación no óptima entre las tablas y generalmente es un indicio de una ausencia de normalización de los datos.

Tablas, Filas, Columnas y Esquemas

Las **tablas** son las estructuras principales de almacenamiento en una base de datos relacional y se organizan en filas y columnas. Cada fila (o tupla) representa un registro único de datos (por ejemplo, la información de un cliente específico), mientras que cada columna representa un atributo del registro (por ejemplo, nombre, dirección, etc.). Un **esquema** es la organización lógica de todas las tablas dentro de una base de datos, que define cómo están estructurados los datos, las relaciones entre las tablas, las restricciones de integridad, y otras reglas.

Diagrama Entidad-Relación (ER)

Un Diagrama Entidad-Relación (ER) es una representación visual de los elementos fundamentales de una base de datos y sus interrelaciones. Este tipo de diagrama se utiliza en el diseño de bases de datos para ilustrar cómo se organizan los datos dentro de cada tabla, qué entidades existen en el modelo de datos, qué atributos tienen estas entidades y cómo se relacionan entre sí. Los componentes de un diagrama ER son:

1. **Entidades:** Representan objetos o conceptos del mundo real que tienen existencia independiente y de los cuales se desea almacenar información en la base de datos. En el diagrama, las entidades se representan mediante rectángulos. Por ejemplo, podríamos tener tablas entidad de Cliente, Producto, Orden, Empleado.
2. **Atributos:** Son las propiedades o características de las entidades. Se representan con óvalos conectados a las entidades correspondientes. Por ejemplo podríamos encontrar los atributos de Nombre, Dirección, Teléfono (atributos de la entidad Cliente).
3. **Relaciones:** Describen la asociación entre dos o más entidades. Las relaciones se representan mediante rombos (o diamantes) que conectan las entidades involucradas. Por ejemplo, podríamos determinar dichas relaciones como asociaciones o acciones tales como "Compra" (relación entre Cliente y Producto), "Empleado-departamento" (relación entre Empleado y Departamento).
4. **Claves Primarias y Claves Foráneas:** Una **clave primaria** es un atributo o un conjunto de atributos que identifica de manera única a una entidad dentro de su tabla. Por otra parte, una **clave foránea** es un atributo o un conjunto de atributos en una entidad que hace referencia a la clave primaria de otra entidad, estableciendo una relación entre ellas.

En muchas ocasiones veremos que algunas bases de datos, permiten entender como los atributos a las columnas de las tablas que representan una entidad fuerte. En adición a esto,

es muy común hacer referencia a las **fact tables** como las tablas principales de la base de datos que contienen los datos transaccionales y corresponden al histórico de registros cuyos campos están traducidos mediante llaves foráneas que apuntan a las **dimension tables,** las cuales contienen la traducción entre un ID (primary key) de un objeto y su categoría correspondiente. Por otra parte, veremos que las **pivot tables** son aquellas tablas que intermedian la conexión de dos tablas generalmente fact y dimension, esto como mecanismo para evitar una relación muchos a muchos, transformándola en una relación uno a muchos entre la tabla pivote y las tablas a las cuales conecta.

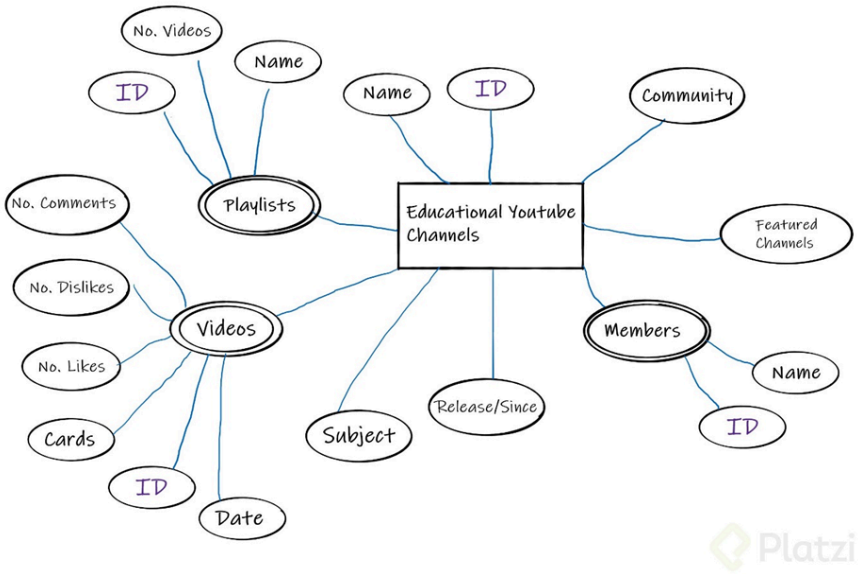


figura 14: Diagrama ER para una base de datos a cerca de canales educativos en YouTube.

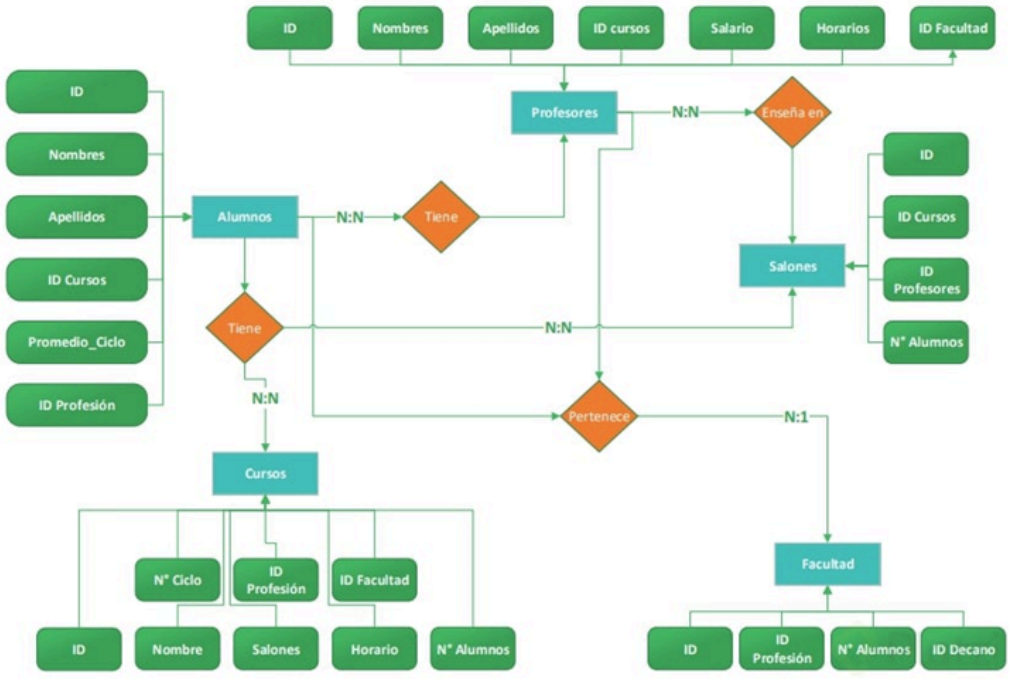


figura 15: Diagrama ER para una base de una universidad.

Normalización en bases de datos

La normalización de una base de datos es un proceso en el diseño de bases de datos SQL que ayuda a organizar los datos de forma que se reduzca la redundancia de datos y se mejore su integridad. Se trata de una serie de directrices y reglas utilizadas para estructurar las tablas de las bases de datos y sus relaciones con el fin de garantizar que la base de datos sea eficiente, fácil de mantener y minimice el riesgo de anomalías en los datos (problemas de coherencia de los datos). Los principales objetivos de la normalización son:

- Eliminar la redundancia de datos.
- Minimizar las anomalías de inserción, borrado y actualización.
- Asegurar la integridad de los datos y hacer que la estructura de la base de datos sea más eficiente.
- Nos ayuda a encontrar formalmente las entidades de nuestra base de datos.

La normalización suele dividirse en diferentes “formas normales”, cada una de las cuales representa un nivel específico de organización e integridad de los datos. Las formas normales más comunes son:

Primera forma normal (1NF): Una tabla está en 1NF si todos los atributos tienen valores atómicos (no divisibles) y no contiene conjuntos o listas de valores. Cada campo debe contener solo un valor único. Consiste en general en asegurar la **no redundancia y atomicidad de la data**, verificando que las tablas cumplan con los siguientes criterios:

- Garantiza que cada columna de una tabla contenga sólo valores atómicos (indivisibles), lo que significa que no hay grupos o matrices de valores que se repitan.
- Elimina las filas duplicadas haciendo que cada fila sea única con una clave primaria.
- El número de columnas debe permanecer invariable y no deben existir columnas repetidas.
- Debe existir una independencia en la disposición de las filas y las columnas. Es decir, si los datos se permutan, el significado de los datos debe ser invariable.

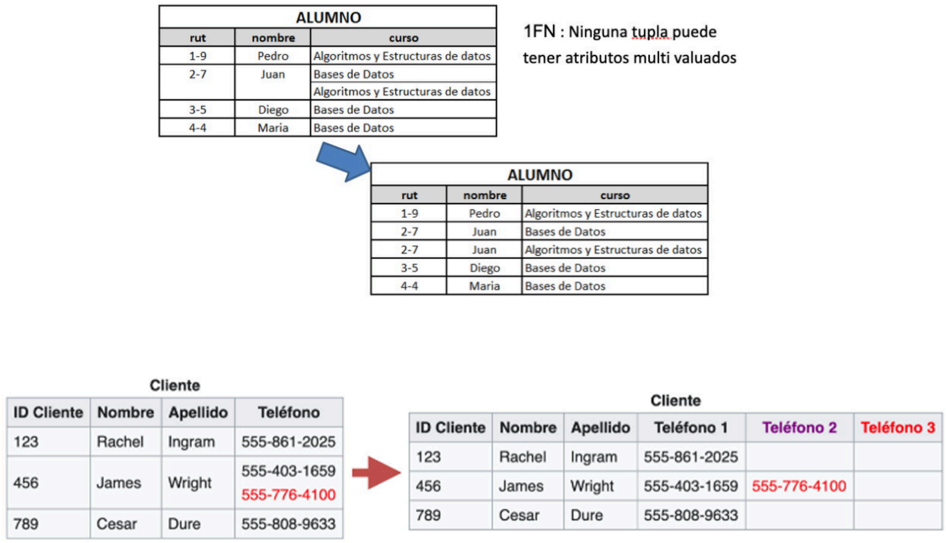


figura 16: Ejemplo de normalización 1NF. En el panel superior, se separan los registros con múltiples valores (en este caso, con respecto a las filas), mientras que en el panel inferior se eliminan duplicados y se separan registros con múltiples valores (en este caso, con respecto a las columnas).

Segunda forma normal (2NF): Una tabla está en 2NF si está en 1NF y todos los atributos no clave son dependientes completamente de la clave primaria. Esto significa que no debe haber dependencias parciales de una parte de la clave primaria (en tablas con claves compuestas). Consiste en general en asegurar la **dependencia funcional entre las tablas a partir de las llaves primarias y la separación de las entidades**, verificando que las tablas cumplan con los siguientes criterios:

- Se debe contar con normalización 1FN.
- Cada fila debe tener un identificador único primario (ID) también conocido como Primary Key.
- Todos los valores de las columnas de las tablas deben depender únicamente de la llave primaria de la tabla.
- Las tablas deben tener una única llave primaria que identifique a la tabla y que sus atributos dependan de ella.

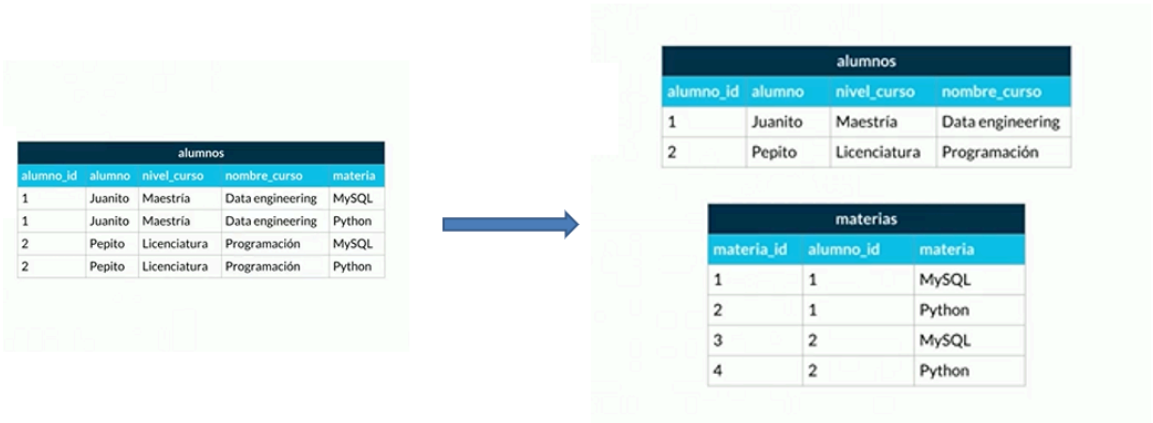
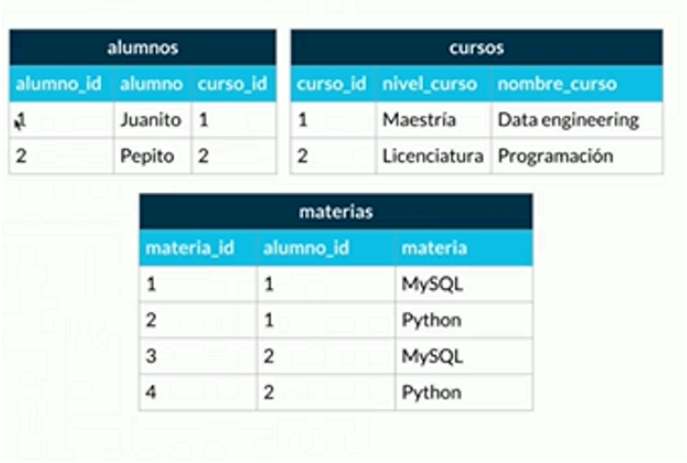


figura 17: Ejemplo de separación de entidades y dependencia con respecto al primary key para asegurar la dependencia funcional entre las tablas.

Tercera forma normal (3NF): Una tabla está en 3NF si está en 2NF y no existen dependencias transitivas entre los atributos. Es decir, ningún atributo no clave debe depender de otro atributo no clave; todos deben depender solo de la clave primaria. Consiste en general en asegurar la **separación de entidades por independencia conceptual**, verificando que las tablas cumplan con los siguientes criterios:

- Se debe contar con normalización 2FN.
- Se debe asegurar que no hay dependencias transitivas, es decir, no deben existir atributos no-llave que dependan entre sí.



alumnos

alumno_id	alumno	curso_id
1	Juanito	1
2	Pepito	2

cursos

curso_id	nivel_curso	nombre_curso
1	Maestría	Data engineering
2	Licenciatura	Programación

materias

materia_id	alumno_id	materia
1	1	MySQL
2	1	Python
3	2	MySQL
4	2	Python

figura 18: Ejemplo de separación conceptual de entidades y dependencia con respecto al primary key.

Cuarta forma normal (4NF): Una versión más estricta de la 3NF, donde cada determinante (columna que determina otra) debe ser una clave candidata. Se utiliza cuando existen dependencias que no cumplen completamente las reglas de 3NF. Permite traducir una tabla transaccional en términos de ID foráneos (en términos de los primary keys de las tablas dimensionales). Consiste en general en asegurar la **unicidad de los regístros de las tablas por separación de entidades por independencia conceptual**, verificando que las tablas cumplan con los siguientes criterios:

- Se debe contar con normalización 3FN.
- Todo atributo no primario debe depender y estar traducido en términos de un atributo primario de una tabla externa.



alumnos

alumno_id	alumno	curso_id
1	Juanito	1
2	Pepito	2

cursos

curso_id	nivel_curso	nombre_curso
1	Maestría	Data engineering
2	Licenciatura	Programación

materias

materia_id	materia
1	MySQL
2	Python

materias_por_alumno

mpa_id	materia_id	alumno_id
1	1	1
2	2	1
3	1	2
4	2	2

Figura 19: Ejemplo de unicidad de registros por separación conceptual de entidades y dependencia con respecto al primary key.

Tipos de datos en bases de datos SQL

En SQL, los tipos de datos definen el tipo de información que se puede almacenar en cada columna de una tabla. Entender los tipos de datos disponibles es crucial para diseñar bases de datos eficientes y garantizar la integridad de los datos. A continuación, se presentan los tipos de datos más comunes que se pueden encontrar en SQL:

Numéricos

- INT, INTEGER: Se utilizan para almacenar números enteros (sin decimales). Dependiendo de la implementación del motor de base de datos, puede tener varios tamaños: 'INT', 'SMALLINT', 'BIGINT'.
- FLOAT, REAL, DOUBLE PRECISION: Son tipos de datos para almacenar números con punto flotante (números decimales). 'FLOAT' y 'REAL' ofrecen diferentes niveles de precisión, siendo 'DOUBLE PRECISION' el de mayor exactitud.
- DECIMAL, NUMERIC: Usados para almacenar números con punto decimal fijo. Son ideales cuando se requiere precisión exacta, como en cálculos financieros. Estos tipos permiten especificar la precisión y escala, es decir, el número total de dígitos y la cantidad de ellos que se encuentran después del punto decimal.

Cadenas de Caracteres

- CHAR(n): Un tipo de dato de longitud fija para cadenas de texto. Se utiliza cuando se sabe que el tamaño del campo será constante. Por ejemplo, 'CHAR(10)' siempre almacenará exactamente 10 caracteres, rellenando con espacios en blanco si es necesario.
- VARCHAR(n): Un tipo de dato de longitud variable que almacena hasta 'n' caracteres. Es más flexible que 'CHAR' porque ocupa espacio solo para el número real de caracteres, hasta el máximo definido.

- TEXT: Utilizado para almacenar cadenas de longitud indefinida. Ideal para grandes cantidades de texto, como descripciones o notas, aunque puede no ser tan eficiente como 'VARCHAR' en términos de rendimiento de consulta.

Fecha y Hora

- DATE: Almacena fechas en el formato 'AAAA-MM-DD'. Es utilizado para guardar datos como fechas de nacimiento, fechas de eventos, etc.
- TIME: Guarda la hora en el formato 'HH:MM:SS'. Ideal para representar horarios específicos dentro de un día.
- TIMESTAMP: Combina la fecha y la hora en un solo valor ('AAAA-MM-DD HH:MM:SS'). Se utiliza para eventos que requieren precisión temporal, como registros de auditoría.
- INTERVAL: Representa un intervalo de tiempo, como días, meses o años. Útil para cálculos de duración o períodos entre fechas.

Booleanos

- BOOLEAN: Representa valores de verdad, típicamente 'TRUE' o 'FALSE'. En algunas implementaciones también se puede utilizar '0' y '1' para representar estos valores.

Binarios

- BYTEA: Se utiliza para almacenar datos binarios de longitud variable, como imágenes, documentos, o cualquier otro archivo binario. 'BYTEA' permite el almacenamiento de datos en formato binario puro.

Tipos de "lenguajes" o usos en SQL

En SQL, los comandos se agrupan en diferentes categorías según su uso específico para manipular y gestionar datos y la estructura de la base de datos. Estos grupos son:

1. **DDL (Data Definition Language):** El Lenguaje de Definición de Datos, corresponde al conjunto de sentencias y prácticas en SQL usadas para definir y alterar la base de datos mediante comandos como 'CREATE', 'ALTER', 'DROP', y 'TRUNCATE', que se utilizan para definir y modificar la estructura de la base de datos (tablas, índices, vistas, esquemas). DDL se centra en la creación, alteración y eliminación de objetos de la base de datos.
2. **DML (Data Manipulation Language):** El Lenguaje de Manipulación de Datos compuesto por sentencias tales como 'SELECT', 'INSERT', 'UPDATE', y 'DELETE', que se utilizan para consultar y manipular los datos contenidos dentro de las tablas. DML se enfoca en la recuperación y actualización de los datos almacenados.
3. **DCL (Data Control Language):** El Lenguaje de Control de Datos, compuesto por sentencias tales como 'GRANT' y 'REVOKE' son utilizados para controlar los permisos y el acceso a los datos y objetos de la base de datos. DCL maneja los aspectos relacionados con la seguridad y autorización.
4. **TCL (Transaction Control Language):** El Lenguaje de Control de Transacciones compuesto por sentencias como 'COMMIT', 'ROLLBACK', y 'SAVEPOINT' para gestionar las

transacciones en la base de datos, asegurando que las operaciones de manipulación de datos sean atómicas, consistentes, aisladas y duraderas (ACID).

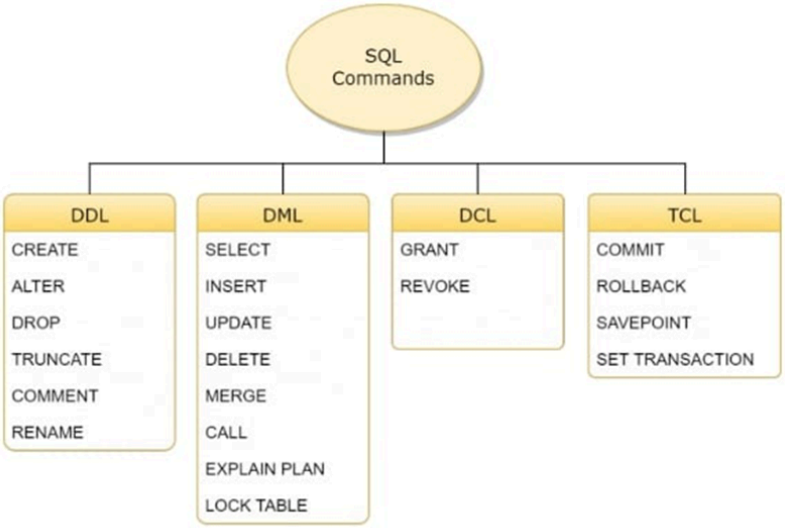


Figura 20: Comandos más usados en SQL, agrupados por su tipo de uso dentro del manejo y definición de bases de datos.

PostgreSQL en VSC

Para el desarrollo de nuestro curso de SQL, usaremos PostgreSQL mediante su interfaz PGAdmin 4 para crear y consultar una base de datos, no obstante también es posible usar integración de Visual Studio Code con SQL para realizar consultas.

PostgreSQL es un sistema de gestión de bases de datos relacional y orientado a objetos que ha ganado reconocimiento por su robustez, flexibilidad y adherencia a los estándares SQL. Desde su lanzamiento inicial en 1996, PostgreSQL ha evolucionado constantemente para incorporar características avanzadas como soporte para tipos de datos personalizados, operaciones transaccionales complejas, manejo de grandes volúmenes de datos y capacidades de extensibilidad. Este enfoque ha hecho que PostgreSQL sea ampliamente considerado como una de las mejores opciones de base de datos de código abierto disponibles en la actualidad.

La importancia de PostgreSQL radica en su capacidad para manejar de manera eficiente operaciones complejas de datos y su solidez en términos de rendimiento y seguridad. Su compatibilidad con múltiples plataformas y su enfoque en el cumplimiento de estándares

SQL lo convierten en una herramienta de elección para muchas organizaciones que buscan una solución de base de datos sin costo de licencia.

Instalación de PostgreSQL

Para poder instalar PostgreSQL en nuestro ordenador, seguimos los pasos siguientes, donde primero debemos dirigirnos a la página oficial de PostgreSQL PostgreSQL y descargarla versión de PostgreSQL compatible con nuestro equipo.

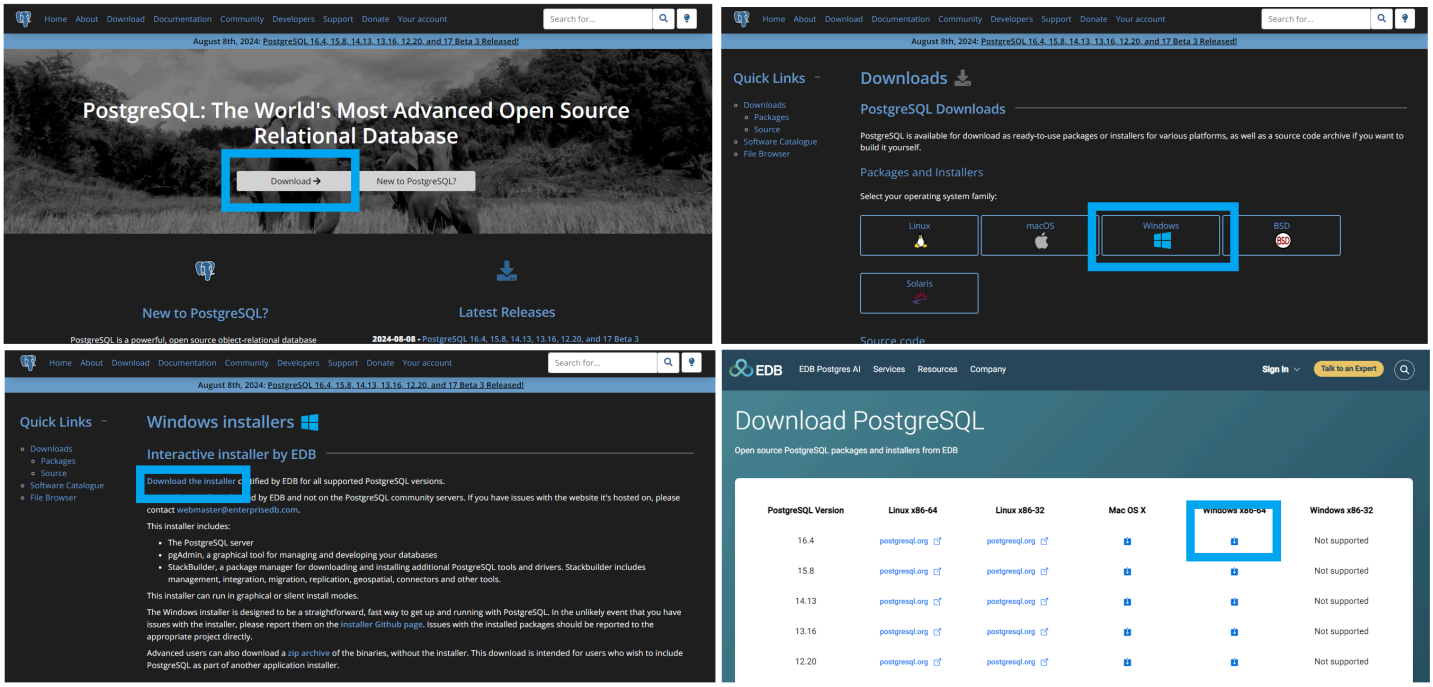


Figura 21: Descarga del instalador de PostgreSQL.

Una vez obtenido el instalador, procedemos a ejecutarlo como cualquier instalador gráfico de Windows, donde en el tercer paso, nos aseguramos que todas las opciones estén seleccionadas, en especial **PGAdmin 4.**

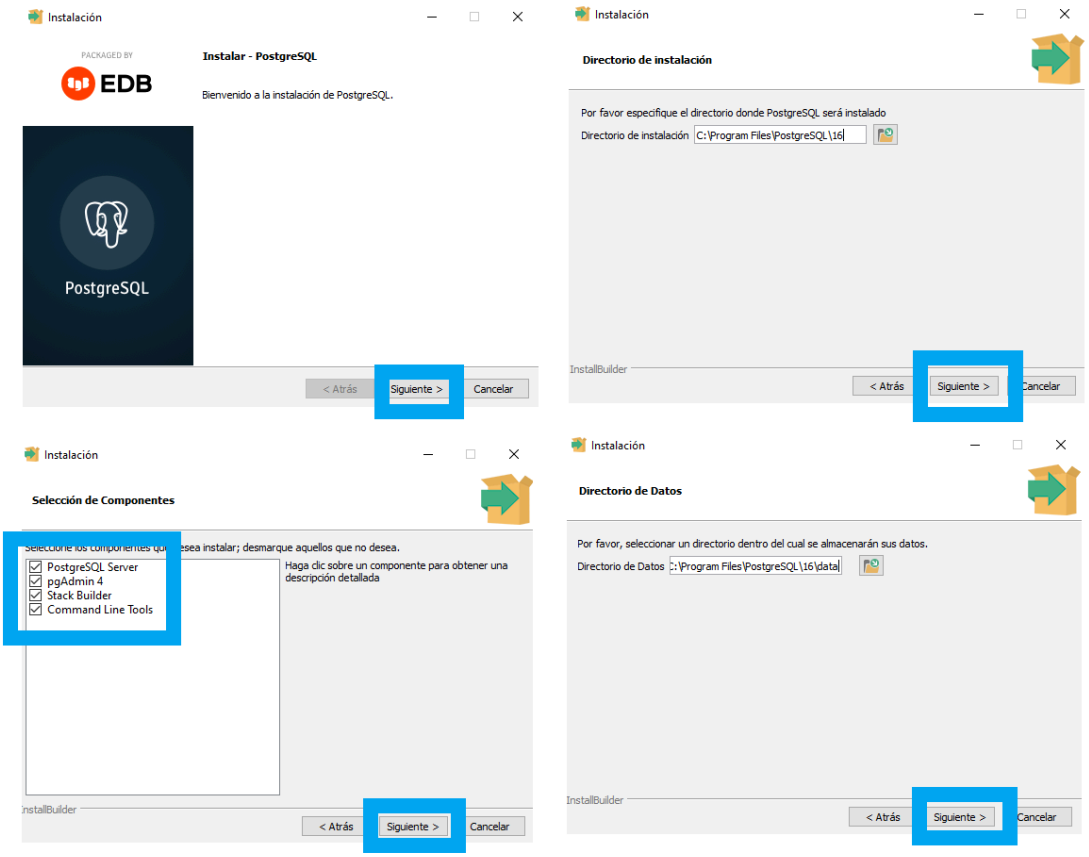


Figura 22: Pasos 1-4 de la instalación de PostgreSQL.

Posteriormente, en el paso 5 y 6 tenemos en cuenta lo siguiente:

- PostgreSQL en el paso 5 nos pedirá una contraseña de usuario, la cual deberemos recordar ya que ésta será la contraseña con la que podamos conectarnos a nuestro servidor local, de olvidarla, no es posible recuperarla y tendremos que empezar de nuevo la instalación.
- En el paso 6 se nos pedirá un número de identificador para realizar las conexiones con nuestro servidor local, por lo general ingresamos **5432** o **5433.**

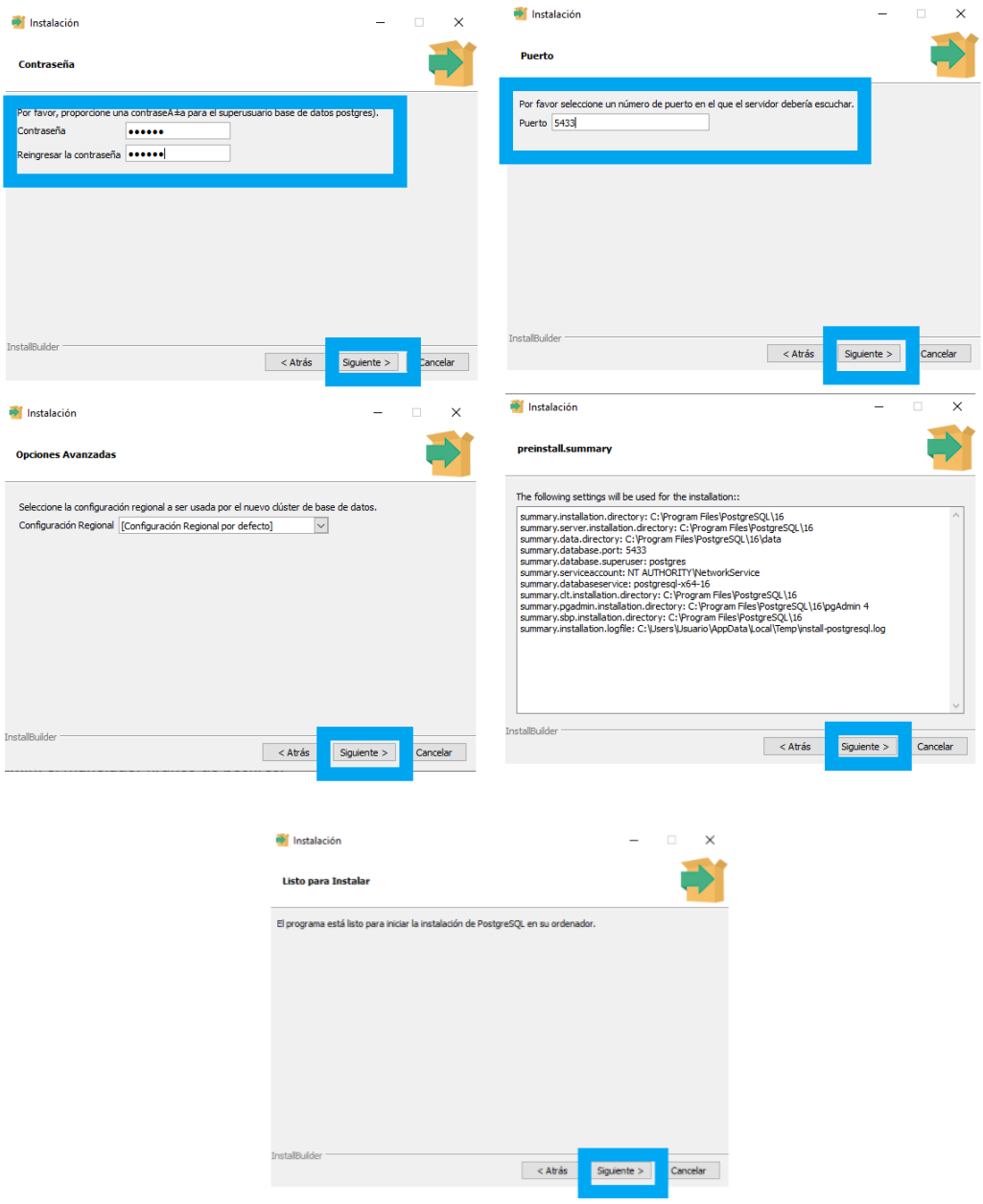


Figura 23: Pasos 5-9 de la instalación de PostgreSQL.

Una vez finalizada la instalación, otro instalador por defecto llamado **Stack Builder** se abrirá intentando instalar automáticamente complementos relacionados con PostgreSQL incluyendo el mísmo PostgreSQL. Tengamos en cuenta lo siguiente ya que este instalador hay que omitirlo:

- En el paso 1 de Stack Builder, seleccionamos la versión de PostgreSQL y el puerto local que instalamos previamente.
- En el paso 2 de Stack Builder seleccionamos en **Database Server** la opción **PostgreSQL** que a un costado dice **(installed).**
- En el paso 3 de Stack Builder, seleccionamos la opción **Next.**
- En el paso 4 de Stack Builder, seleccionamos la opción **Skip installation.** Lo cual nos sacará de Stack Builder y la instalación de PostgresSQL no será repetida. Finalmente en el paso 5 seleccionamos **Finish.**

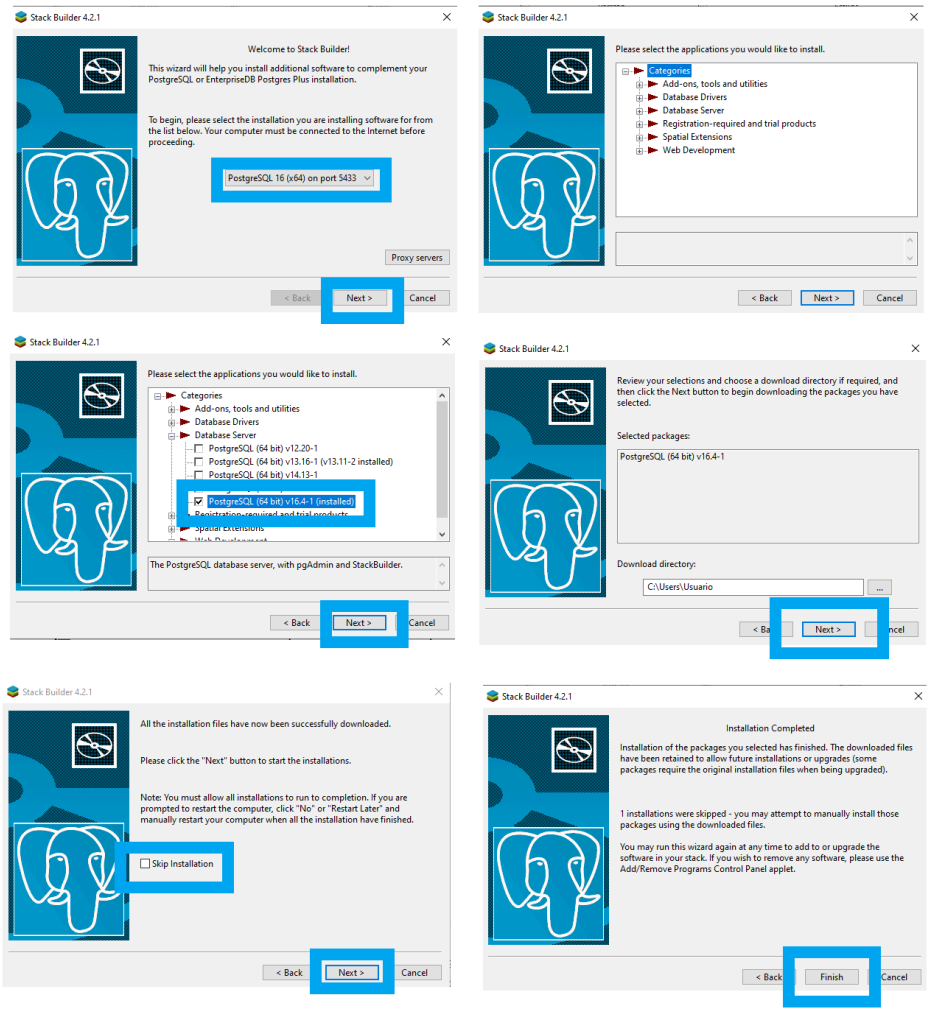


Figura 24: Omisión de la instalación de PostgresQL por medio de Stack Builder.

PGAdmin 4 es una plataforma gráfica para el uso de PostgreSQL que automáticamente genera una conexión inicial al manejador de base de datos denominada “posgres” , o “PosgreSQL 16” (el número al final cambia según la versión instalada). Cada vez que seleccionemos una conexión, nos solicitará ingresar la clave con la cual la conexión ha sido establecida, en este caso al ser la conexión local, deberemos ingresar la clave que ingresamos durante la instalación.

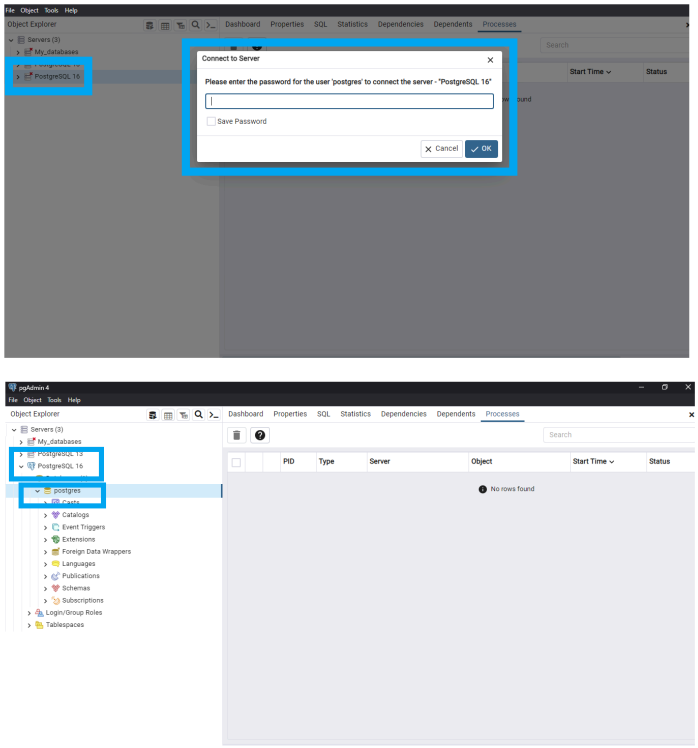


Figura 25: Conexión al servidor local desde PGAdmin 4.

Para poder usar PostgreSQL dentro de VSC, nos dirigimos a la sección de extensiones y buscamos **SQLTools , SQLTools PostgresSQL/Cockroach Driver** y procedemos a instalarlas.

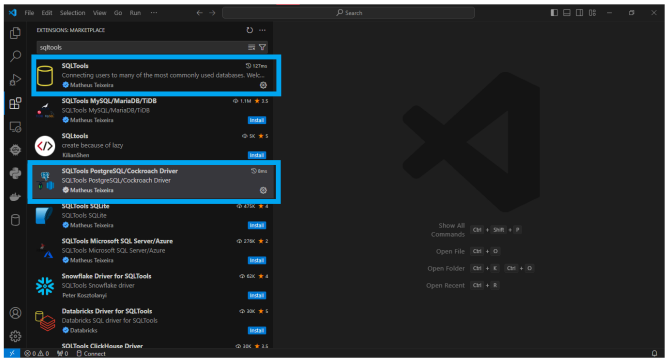


Figura 26: Instalación de las extensiones necesarias de VSC para poder usar PostgresSQL dentro de VSC.

Para poder generar una conexión local de PostgreSQL desde VSC, seleccionamos en el panel izquierdo le ícono de SQLTools, luego, seleccionamos la opción “agregar conexión” y seleccionamos la opción “PostgreSQL”. En el paso dos, especificamos los siguientes valores iniciales:

- **Connection name:** PostgreSQL 16
- **Connecting using:** Server and Port
- **Server adress:** localhost
- **Port:** 5433
- **Database:** postgres
- **Username:** postgres
- **Use pasword:** Save as plain text in setting
- **Pasword:** Digita la contraseña de instalación

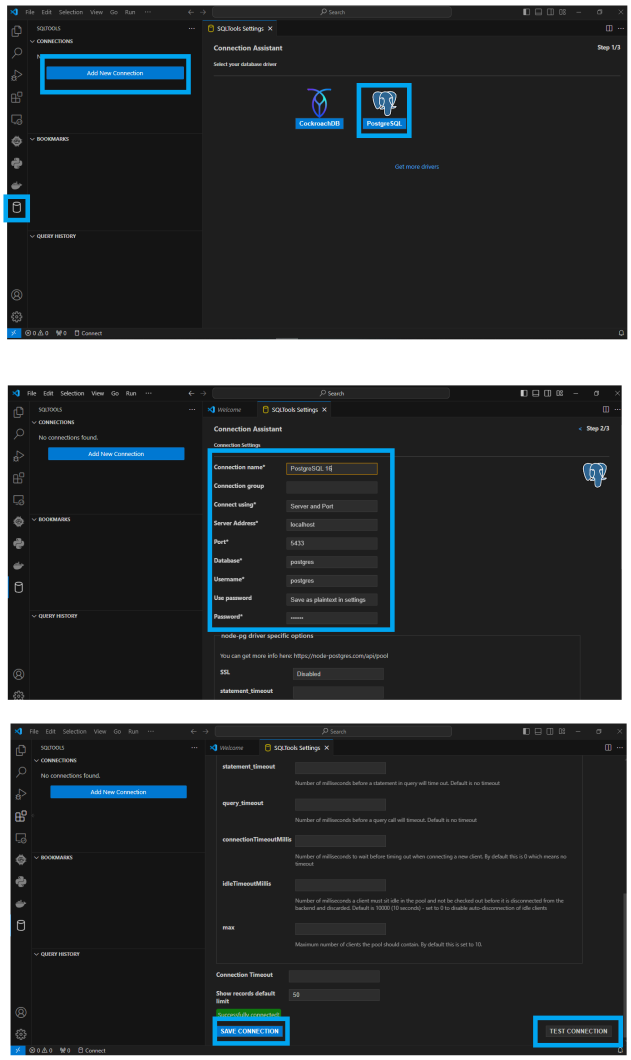


Figura 27: Creación de una conexión local de PostgreSQL en VSC.

Data Definition Language en SQL

Dentro de SQL, DDL (Data Definition Language) es una subcategoría del lenguaje SQL que se utiliza para definir y administrar la estructura de las bases de datos y sus objetos, como tablas, índices, vistas, y esquemas. A través de sentencias DDL, los administradores de bases de datos pueden crear, modificar o eliminar elementos de la estructura de la base de datos, estableciendo así el "esqueleto" que soporta el almacenamiento y manejo de datos.

Sentencia CREATE

Creación de una base de datos simple

```
--------------------------------------
--Creación de una base de datos con DDL
--------------------------------------

CREATE DATABASE medical_db;
```

Creación de las tablas de entidades fuertes

```
--tabla pacientes
CREATE TABLE patients(
    ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    name TEXT NOT NULL,
    adress VARCHAR(40),
    phone VARCHAR(30),
    date_of_birth DATE
);

--tabla médicos
CREATE TABLE physicians(
    ID INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    name TEXT NOT NULL,
    specialty TEXT,
    licence_number TEXT UNIQUE
);
```

Creación de las tablas de entidades débiles y de las relaciones dentro del esquema

```
--tabla citas
CREATE TABLE appointments(
    ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    patient_id BIGINT REFERENCES patients (ID),
    physician_id BIGINT REFERENCES physicians (ID),
    appointment_datetime TIMESTAMP WITH TIME ZONE
);

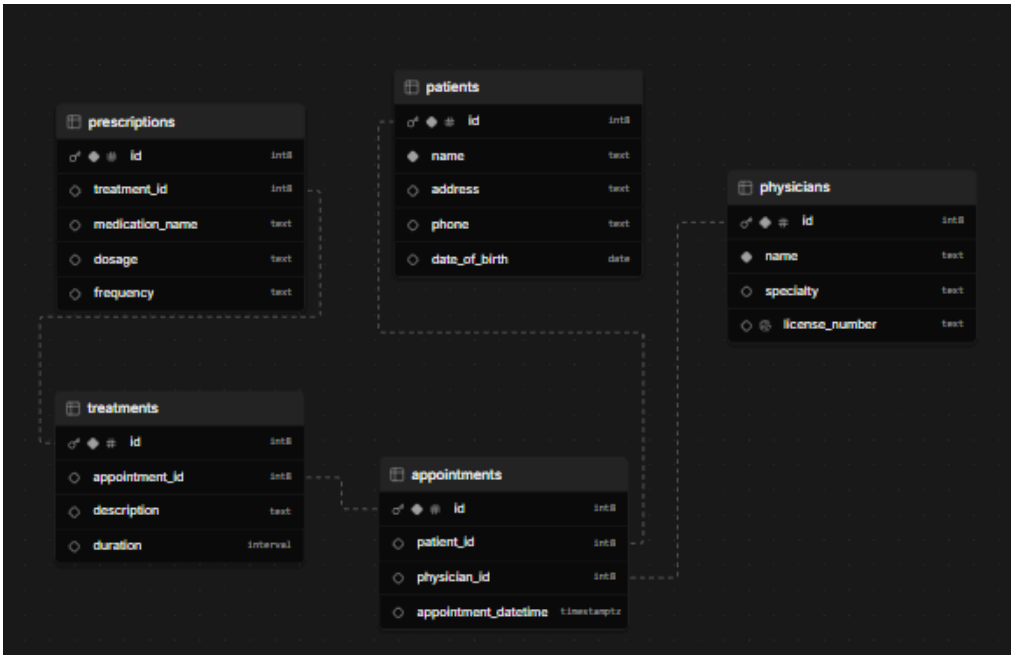
--tabla tratamientos
CREATE TABLE treatments(
    ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    appointment_id BIGINT REFERENCES appointments (ID),
    description TEXT,
    duration INTERVAL
);

--tabla prescripciones
CREATE TABLE prescriptions(
    ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    treatment_id BIGINT REFERENCES treatments (ID),
    medication_name TEXT NOT NULL,
    dosage TEXT,
    frequency TEXT
);
```

```sql
--tabla aseguradoras
CREATE TABLE insurance_companies(
    ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    name VARCHAR(50) NOT NULL,
    contact_number VARCHAR(50),
    adress TEXT
);

--tabla hospitales
CREATE TABLE hospitals(
    ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    name VARCHAR(40) NOT NULL,
    location TEXT,
    contact_number VARCHAR(50)
);
```

Con las tablas hasta ahora generadas, el esquema relacional resultante hasta el momento luce de la siguiente manera.



Sentencia ALTER y DROP

No obstante la base de datos no esta lista para ser usada. Para realizar unas modificaciones en la base de datos, empleamos la sentencia **ALTER TABLE table_name** para agregar una columna tipo **BIGINT** en las tablas **patients** y **physicians.**

```sql
--agregamos la columna insurance_id a patients
ALTER TABLE patients
ADD COLUMN insurance_id BIGINT REFERENCES insurance_companies(ID);

--agregamos la columna hospital_id a physicians
ALTER TABLE physicians
ADD COLUMN hospital_id BIGINT REFERENCES hospitals(ID);
```

ALTER nos permite modificar nuestras tablas especificando en la linea siguiente a la especificación de la tabla de interés, el tipo de modificación que queremos hacer. Por ejemplo, agregaremos dos columnas nuevas a **patients,** para posteriormente renombrar una columna de **insurance_companies** y **patients.**

```
--agregamos la columna gender a patients e email
ALTER TABLE patients
ADD COLUMN gender VARCHAR(10);

ALTER TABLE patients
ADD COLUMN email TEXT;
```

```
--corregimos el nombre de la columna adress de insurance companies y patients
ALTER TABLE insurance_companies
RENAME COLUMN adress TO address;

ALTER TABLE patients
RENAME COLUMN adress TO address;
```

Por otra parte, otro uso común de la sentencia ALTER es cambiar el tipo de datos en cada columna. Por ejemplo, en la definición de nuestras tablas, hemos decidido usar la combinación de sentencias **ID BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY** para generar un identificador incremental que preserve la identidad del registro en su ID independientemente de si algún registro es eliminado más adelante. Supongamos ahora que por conveniencia de nuestro ejercicio, sabemos que tendremos que insertar datos y eliminarlos constantemente pero no nos interesa que cada dato preserve un contador único e irrepetible sino que solo nos interesa un identificador incremental que tenga en cuenta única y exclusivamente los datos presentes actualmente en la base de datos y no aquellos que fueron eliminados. Para ello, empleamos las siguientes sentencias en combinación con ALTER.

No obstante, ya que nuestro objetivo es cambiar la naturaleza de los IDs principales los cuales poseen relaciones definidas a lo largo de las tablas, el motor de base de datos nos obligará a ejecutar los siguientes pasos:

- Eliminar las relaciones.
- Eliminar las columnas de ID.
- Generar una nueva columna ID para cada tabla.
- Estipular un objeto serial autoincremental para el ID de cada tabla.
- Definir las llaves primarias y foráneas para establecer una relación entre estas.

```
96  ----------------------------------------
97  --Eliminación de relaciones entre tablas
98  ----------------------------------------
99
100 --intentamos eliminar una relacion
101 ALTER TABLE insurance_companies
102 DROP COLUMN ID;
103
```

Data Output | Messages | Notifications

```
ERROR:  restricción «patients_insurance_id_fkey» en tabla patients depende de columna id de tabla insurance_companiesno se puede
eliminar columna id de tabla insurance_companies porque otros objetos dependen de él

ERROR:  no se puede eliminar columna id de tabla insurance_companies porque otros objetos dependen de él
SQL state: 2BP01
Detail: restricción «patients_insurance_id_fkey» en tabla patients depende de columna id de tabla insurance_companies
Hint: Use DROP ... CASCADE para eliminar además los objetos dependientes.
```

Así pues, primero identificaremos el nombre de la relación que postgres reconoce dentro de la base de datos. Para poder eliminar cualquier cosa en SQL nos valemos de la sentencia **DROP.** Sin embargo, el hecho de querer eliminar una relación implica que esterémos alterando toda la base de datos gracias a que las relaciones son el componente fundamental de la comunicación entre tablas, por ello, si simplemente ejecutamos una sentencia DROP, la lógica de SQL nos impedirá hacerlo gracias a que otras tablas se ven involucradas en el drop. Para poder especificarle a postgres que en efecto esto es lo que queremos, y que deseamos alterar el esquema teniendo en cuenta todos los posibles objetos afectados, empleamos la sentencia **CASCADE.** Esta sentencia es una opción que se utiliza en comandos SQL para indicar que cuando se elimina o cambia una tabla o una restricción (especificada por la sentencia **CONSTRAINT**), todos los objetos que dependen de ella también deben ser eliminados o actualizados automáticamente.

```
104 --identificamos el nombre de las restricciones para despues eliminarlas
105 SELECT
106     constraint_name
107 FROM
108     information_schema.key_column_usage
109 WHERE
110     table_name = 'insurance_companies'
111     AND column_name = 'id';
112
113
```

Data Output | Messages | Notifications

	constraint_name name
1	insurance_companies_pkey

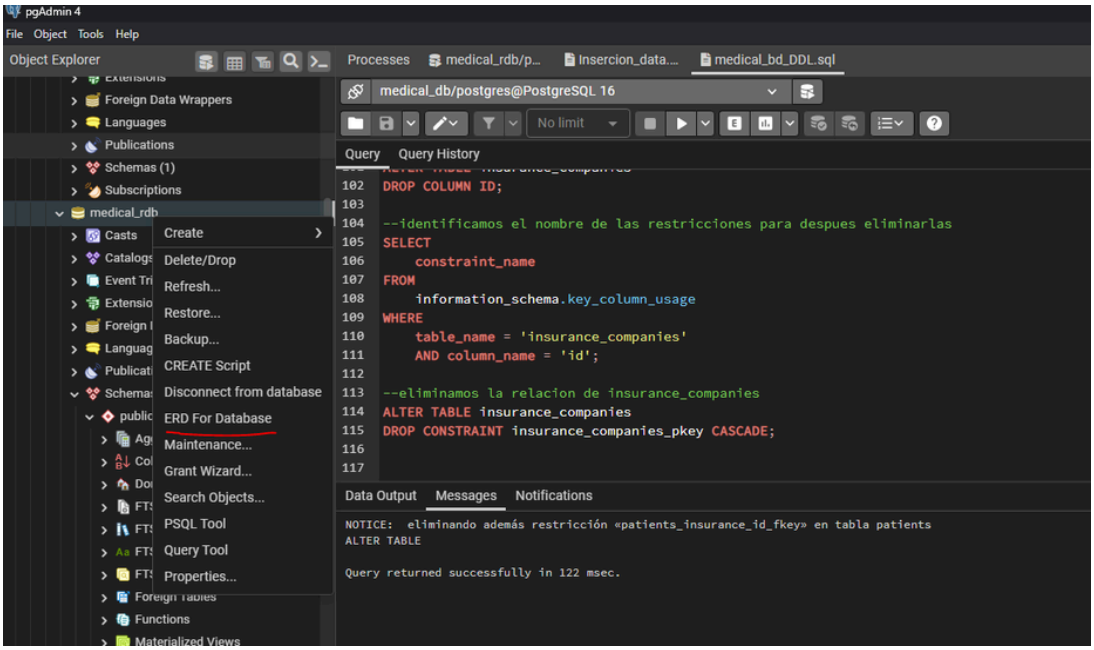
```
113 --eliminamos la relacion de insurance_companies
114 ALTER TABLE insurance_companies
115 DROP CONSTRAINT insurance_companies_pkey CASCADE;
116
117
```

Data Output | Messages | Notifications

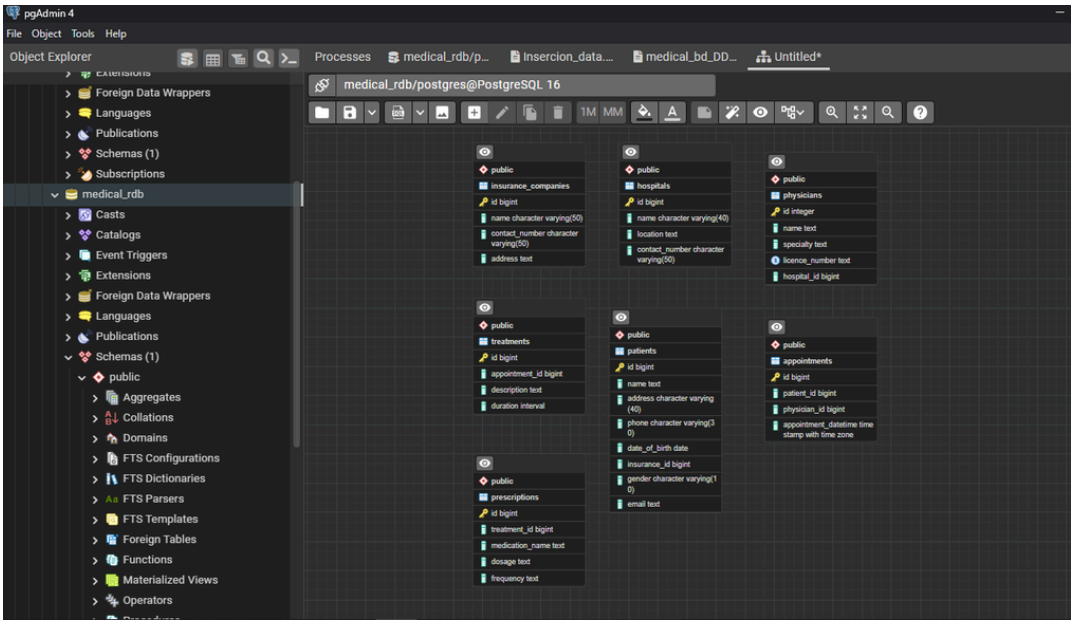
```
NOTICE:  eliminando además restricción «patients_insurance_id_fkey» en tabla patients
ALTER TABLE

Query returned successfully in 122 msec.
```

Ejecutamos este procedimiento para cada tabla y observamos ahora el diagrama ER. Dentro de pgAdmin4 existe una forma de ver dicho esquema, simplemente hacemos click derecho en nuestra base de datos y seleccionamos la opción que dice **EDR for Database.**



En donde si hemos eliminado las relaciones correctamente, nuestro diagrama ER luciría de la siguiente manera.



Ahora, empleamos la sentencia conjunta **ALTER TABLE** y **ALTER COLUMN** para eliminar el atributo **IDENTITY** el cual no permite restaurar los IDs de los datos sino que preserva la identidad del ID a lo largo del tiempo a pesar de que algunos datos hayan sido eliminados. Ejecutamos las siguientes sentencias para todas las tablas.

```
136 ------------------------------------
137 --Eliminación del atributo IDENTITY
138 ------------------------------------
139
140 ALTER TABLE insurance_companies
141 ALTER COLUMN id DROP IDENTITY;
142
143 ALTER TABLE appointments
144 ALTER COLUMN id DROP IDENTITY;
145
```

Una vez eliminada la restricción, generamos un objeto serial para autoincrementar el ID y asignamos dicho objeto serial al identificador de llave primaria de cada una de nuestras tablas.

```
--generamos una serie la cual nos va a servir para
--incrementar automaticamente los IDs

CREATE SEQUENCE id_serial
START WITH 1
INCREMENT BY 1;

--asignamos la serie del identificador a cada columna
--ID de nuestras tablas

ALTER TABLE insurance_companies
ALTER COLUMN id SET DEFAULT nextval('id_serial');

ALTER TABLE appointments
ALTER COLUMN id SET DEFAULT nextval('id_serial');
```

Finalmente, nos aseguramos de que el ID sea el identificador primario de cada uno de las tablas que tenemos en la base de datos. Este paso es importante debido a que SQL no nos permitirá establecer una relación entre un foregein key y la llave primaria de una tabla a menos que indiquemos de antemano que la columna de ID a la cual estamos haciendo referencia, posea la característica de identificador único, propiedad implícita dentro de la definición de un primary key.

```
--nos aseguramos de que primary key sea el ID en cada tabla
ALTER TABLE insurance_companies
ADD CONSTRAINT insurance_companies_pkey PRIMARY KEY (id);

ALTER TABLE appointments
ADD CONSTRAINT appointments_pkey PRIMARY KEY (id);

ALTER TABLE hospitals
ADD CONSTRAINT appointments_pkey PRIMARY KEY (id);
```

No obstante, a pesar de que hemos cambiado la naturaleza de nuestros IDs, si realizamos una consulta veremos que los IDs aún preservan su valor. En este sentido, emplearemos la sentencia **TRUNCATE** para poder eliminar todos los registros incorrectamente ingresados en nuestra base de datos.

Data Output | Messages | Notifications

	id [PK] bigint	name character varying (50)	contact_number character varying (50)	address text
1	13	HealthFirst	123-456-7890	123 Health St
2	14	WellCare	234-567-8901	456 Wellness Ave
3	15	UnitedHealth	345-678-9012	789 United Blvd

```
-----------------------------------------------------------------
--Eliminación de los datos de las tablas incorrectamente ingresados
-----------------------------------------------------------------

TRUNCATE TABLE insurance_companies RESTART IDENTITY;
TRUNCATE TABLE appointments RESTART IDENTITY;
TRUNCATE TABLE hospitals RESTART IDENTITY;
TRUNCATE TABLE patients RESTART IDENTITY;
TRUNCATE TABLE physicians RESTART IDENTITY;
TRUNCATE TABLE prescriptions RESTART IDENTITY;
TRUNCATE TABLE treatments RESTART IDENTITY;
```

Sentencia PRIMARY KEY, FOREIGN KEY y REFERENCES

Una vez corregida nuestra base de datos, procedemos a agregar de nuevo las relaciones que definirán el esquema relacional de la base de datos. Para ello, nos aseguramos primero de que las columnas ID de cada tabla hallan sido definidas como los **PRIMARY KEY.** Luego, nos aseguramos de eliminar cualquier constricción remanente posible dentro de las llaves foráneas de nuestras tablas para evitar conflictos.

```sql
--nos aseguramos de eliminar cualqueir restriccion sobre el foregein key
ALTER TABLE patients
DROP CONSTRAINT IF EXISTS patients_insurance_id_fkey;

ALTER TABLE physicians
DROP CONSTRAINT IF EXISTS physicians_hospital_id_fkey;

ALTER TABLE appointments
DROP CONSTRAINT IF EXISTS appointments_patient_id_fkey;

ALTER TABLE appointments
DROP CONSTRAINT IF EXISTS appointments_physician_id_fkey;
```

Finalmente, agregamos la relación como una combinación entre las sentencias **ALTER TABLE, ADD CONSTRAINT** y **FOREIGN KEY, REFERENCES.** La sentencia FOREIGN KEY permite definir la columna dentro de nuestra tabla mediante la cual haremos referencia a otra tabla foránea, mientras que la sentencia REFERENCES es empleada para en efecto, indicar a la base de datos que la llave foránea referenciará (estará conectada) a una llave principal ubicada en otra tabla.

```sql
-- recreamos las relaciones entre los primary y foregein keys
ALTER TABLE patients
ADD CONSTRAINT patients_insurance_id_fkey FOREIGN KEY (insurance_id) REFERENCES insurance_companies (id);

ALTER TABLE physicians
ADD CONSTRAINT physicians_hospital_id_fkey FOREIGN KEY (hospital_id) REFERENCES hospitals (id);

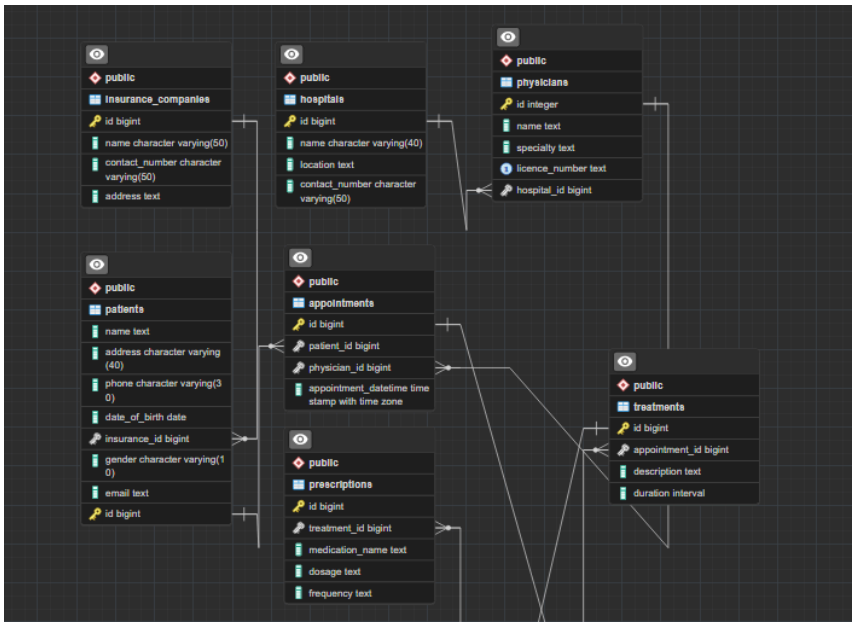
ALTER TABLE appointments
ADD CONSTRAINT appointments_patient_id_fkey FOREIGN KEY (patient_id) REFERENCES patients (id);

ALTER TABLE appointments
ADD CONSTRAINT appointments_physician_id_fkey FOREIGN KEY (physician_id) REFERENCES physicians (id);

ALTER TABLE treatments
ADD CONSTRAINT treatments_appointment_id_fkey FOREIGN KEY (appointment_id) REFERENCES appointments (id);

ALTER TABLE prescriptions
ADD CONSTRAINT prescriptions_treatment_id_fkey FOREIGN KEY (treatment_id) REFERENCES treatments (id);
```

El código anterior, agrega las relaciones necesarias de manera correcta, generando un esquema ER de la siguiente forma.



Data Manipulation Language en SQL

Dentro de SQL, el DML (Data Manipulation Language) es una subcategoría del lenguaje SQL que se utiliza para manipular datos en una base de datos. A través del DML, los usuarios pueden realizar operaciones para insertar, actualizar, eliminar y consultar datos almacenados en las tablas de una base de datos. Las sentencias DML más usadas son

- **INSERT:** Sirve para insertar nuevas filas en una tabla. Su uso se define en conjunto a la sentencia INTO y VALUES.
- **UPDATE:** Permite actualizar los datos existentes en una tabla. Su uso se define en conjunto con las sentencias SET y WHERE.
- **DELETE:** Se usa para eliminar filas de una tabla.
- **SELECT:** Se utiliza para consultar y recuperar datos de una o más tablas. Su uso más común, se define en conjunto con las sentencias FROM y las variaciones de la sentencia JOIN.
- **FROM:** Indica la fuente de datos sobre la cual se quiere realizar la consulta.

Sentencia INSERT

La sentencia **INSERT** en SQL permite agregar una o más filas de datos a una tabla. Esta operación puede llevarse a cabo en una tabla con todos los valores de las columnas o solo en columnas específicas. Su sintaxis básica es la siguiente, donde **INSERT INTO** especifica la tabla y columnas en las cuales ingresaremos información, mientras que **VALUES** permite especificar los valores a insertar.

```sql
--insertar datos en una tabla
INSERT INTO tabla (columna1, columna2)
VALUES (valor1, valor2)
```

Dentro del material de nuestro curso, los datos correspondientes a cada tabla se encuentran en archivos SQL bajo el nombre **tabla_data.sql**, los cuales son archivos de texto SQL con la estructura de código que vemos en la imagen anterior donde cada insert en una tabla, corresponde a un único registro.

Sentencia UPDATE y WHERE

La sentencia **UPDATE** en SQL se utiliza para modificar los datos de una o más filas en una tabla existente. Esta sentencia permite cambiar el valor de una o varias columnas en las filas que cumplen con una condición específica. Si no se usa una condición (**WHERE**), se actualizarán todas las filas de la tabla. La principal utilidad de UPDATE es cambiar valores que ya existen en una tabla. Por ejemplo, si el número de teléfono de un cliente ha cambiado, se puede actualizar fácilmente con esta sentencia. Si sintaxis básica viene dada por:

```sql
UPDATE nombre_tabla
SET columna1 = valor1, columna2 = valor2, ...
WHERE condicion;
```

Sentencia SELECT, uniones JOIN y funciones de agrupación

La sentencia **SELECT** en SQL es la instrucción más utilizada para **consultar y recuperar datos** de una o varias tablas de una base de datos. La **estructura básica** del comando SELECT permite seleccionar columnas específicas o todos los datos de una tabla, aplicar filtros, ordenar resultados, realizar agrupamientos, y otras operaciones complejas de manipulación de datos.

```sql
SELECT column1, column2, ...
FROM table_name;
```

Si se desea seleccionar toda la información de una tabla se emplea la sentencia **SELECT * FROM table_name;** no obstante, existe una variedad enorme de formas de consulta. Veamos las más básicas:

- **Selección mediante condición de filtro**

```sql
SELECT t.columna1, t.columna2
FROM table AS t
WHERE condition
```

- **Selección organizada por característica**

```sql
SELECT column1, column2
FROM table
ORDER BY column_to_order DESC;
```

- **Selección de valores únicos**

```sql
SELECT DISTINCT column1
FROM table;
```

- **Selección de N primeros elementos con y sin offset**

```sql
SELECT columna1, columna2, columna3,
FROM table
LIMIT n

SELECT columna1, columna2, columna3,
FROM table
LIMIT n OFFSET m
```

- **Selección de elementos agrupados y conteo de agrupación**

```sql
SELECT columna1, columna2, columna3, COUNT(*)
FROM table
GROUP BY column_to_group;
```

Los **joins** en SQL son una herramienta fundamental para combinar datos de dos o más tablas en una base de datos. En situaciones donde la información relevante está distribuida en varias tablas, los joins permiten extraer y organizar los datos relacionados de manera eficiente, integrando la información en un solo conjunto de resultados. Un join es una operación que combina filas de dos o más tablas basándose en una columna común entre ellas. Esta columna puede ser una clave primaria en una tabla y una clave foránea en otra, lo que garantiza que exista una relación entre las tablas. Los joins son esenciales para trabajar con bases de datos relacionales, ya que los datos suelen estar distribuidos entre varias tablas para reducir la redundancia y mejorar la eficiencia.

Existen varios tipos de joins, cada uno con características específicas que determinan cómo se combinan los datos y qué registros se incluyen en el resultado final. Los tipos principales son:

INNER JOIN (Unión interna): El INNER JOIN es el tipo más común y devuelve solo las filas donde hay coincidencia en ambas tablas. Si una fila de una tabla no tiene correspondencia en la otra, no se incluirá en el resultado. Su sintaxis básica es:

```sql
SELECT columnas
FROM tabla1
INNER JOIN tabla2
ON tabla1.columna_comun = tabla2.columna_comun;
```

LEFT JOIN (Unión izquierda): El LEFT JOIN devuelve todas las filas de la tabla de la izquierda (tabla1), junto con las filas coincidentes de la tabla de la derecha (tabla2). Si no hay coincidencia en la tabla derecha, se devolverán `NULL` para las columnas de esa tabla.

```sql
SELECT columnas
FROM tabla1
LEFT JOIN tabla2
ON tabla1.columna_comun = tabla2.columna_comun;
```

RIGHT JOIN (Unión derecha): El RIGHT JOIN es el inverso del LEFT JOIN. Devuelve todas las filas de la tabla de la derecha, junto con las filas coincidentes de la tabla de la izquierda. Si no hay coincidencia en la tabla izquierda, se devolverán **NULL** en las columnas de esa tabla.

```sql
SELECT columnas
FROM tabla1
RIGHT JOIN tabla2
ON tabla1.columna_comun = tabla2.columna_comun;
```

FULL JOIN o FULL OUTER JOIN (Unión externa completa): El FULL JOIN combina el comportamiento de los LEFT JOIN y RIGHT JOIN. Devuelve todas las filas cuando hay coincidencias en cualquiera de las tablas, y para las filas sin coincidencia en cualquiera de las tablas, se devolverán **NULL**.

```sql
SELECT columnas
FROM tabla1
FULL JOIN tabla2
ON tabla1.columna_comun = tabla2.columna_comun;
```

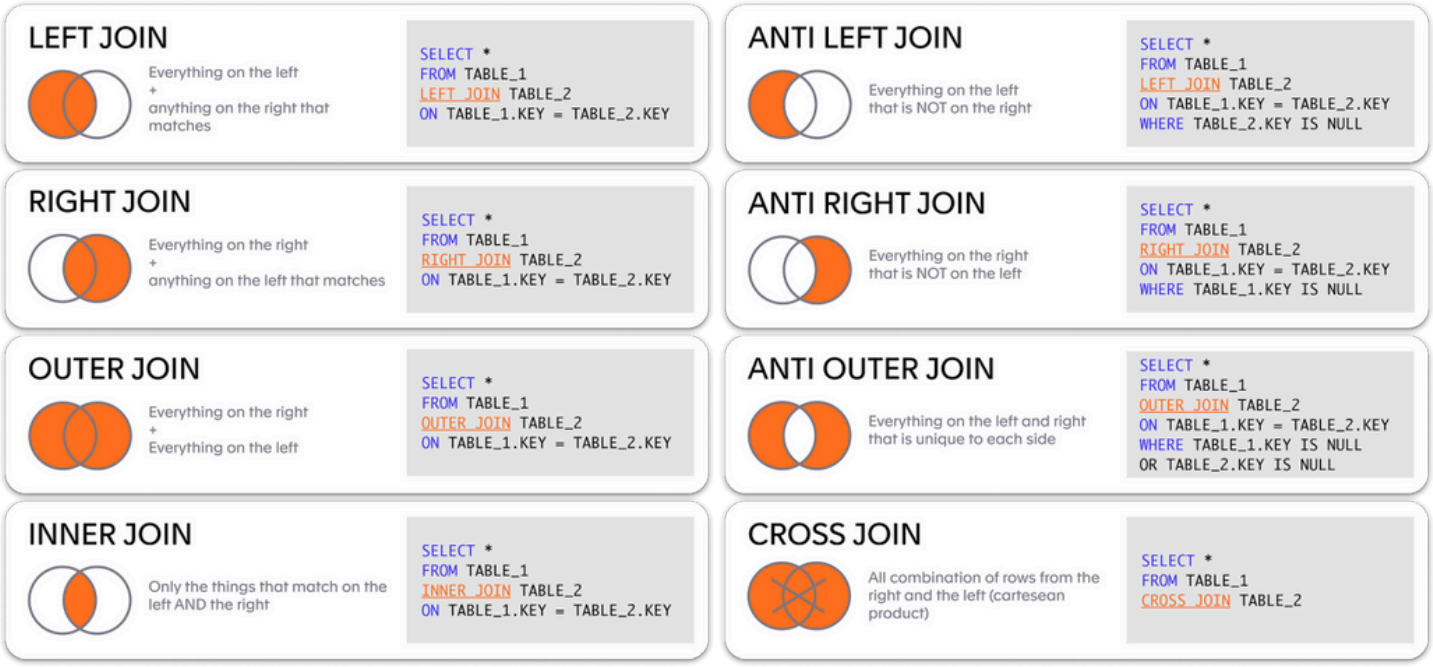
CROSS JOIN (Unión cruzada): El CROSS JOIN devuelve el producto cartesiano de las dos tablas. Es decir, combina cada fila de la primera tabla con cada fila de la segunda tabla, lo que puede generar un gran número de combinaciones, dependiendo del tamaño de las tablas.

```sql
SELECT columnas
FROM tabla1
CROSS JOIN tabla2;
```

SELF JOIN (Auto-unión): El SELF JOIN es una operación donde una tabla se une consigo misma. Esto puede ser útil cuando se necesita comparar filas dentro de la misma tabla. Para hacer un self join, se utilizan **alias** para distinguir las diferentes instancias de la misma tabla.

```sql
SELECT a.columna1, b.columna2
FROM tabla AS a
JOIN tabla AS b
ON a.columna_comun = b.columna_comun;
```

La imagen a continuación ilustra no solo la sintaxis de cada uno de los tipos de uniones sino que muestra en términos de los diagramas de Venn, como se puede interpretar cada unión de tablas.



Las **funciones de agrupación** en SQL son esenciales para realizar cálculos agregados sobre conjuntos de datos. Estas funciones toman un conjunto de valores como entrada y devuelven un solo valor representativo, lo que es especialmente útil para generar resúmenes de datos. A continuación, se describen las funciones de agrupación más comunes en SQL, con una explicación detallada de cada una:

- **COUNT():** Calcula el número total de filas que coinciden con una condición o en una columna específica. Es ideal para contar filas sin importar si los valores son nulos o no. Puede contar todas las filas o solo aquellas que contienen un valor no nulo en una columna específica.

```
SELECT COUNT(*) FROM table; -- Cuenta todas las filas
SELECT COUNT(columna) FROM table; -- Cuenta las filas donde 'columna' no es NULL
```

- **SUM():** Calcula la suma total de los valores de una columna numérica. Se utiliza para sumar los valores de una columna. Si hay valores nulos en la columna, SUM() los ignora.

```
SELECT SUM(columna) FROM table; -- Suma todos los valores numéricos de la columna
```

- **AVG():** Calcula el valor promedio de una columna numérica. Ignora valores nulos y calcula la media aritmética de los valores.

```
SELECT AVG(columna) FROM table; -- Calcula el promedio de los valores en la columna
```

- **MIN()**: Devuelve el valor mínimo de una columna. Se usa para encontrar el valor más bajo en una columna, ya sea numérica, de texto o fecha.

```
SELECT MIN(columna) FROM table; -- Encuentra el valor mínimo de la columna
```

- **MAX():** Devuelve el valor máximo de una columna. Se utiliza para encontrar el valor más alto en una columna, ya sea numérica, de texto o fecha.

```sql
SELECT MAX(columna) FROM table; -- Encuentra el valor máximo de la columna
```

- **GROUP_CONCAT()* (o `STRING_AGG()` en PostgreSQL):** Devuelve una cadena que es la concatenación de los valores de un grupo. Es útil cuando quieres concatenar valores de una columna en una sola cadena, separando los valores por comas u otros delimitadores.

```sql
SELECT GROUP_CONCAT(columna SEPARATOR ', ') FROM table; -- MySQL
SELECT STRING_AGG(columna, ', ') FROM table; -- PostgreSQL
```

- **VARIANCE() (Varianza):** Calcula la varianza de un conjunto de datos numéricos, que mide la dispersión de los datos respecto al promedio. La varianza es útil en estadísticas y es una medida de la variabilidad de los valores.

```sql
SELECT VARIANCE(columna) FROM table;
```

- **STDDEV() (Desviación estándar):** Calcula la desviación estándar, que mide la cantidad de variación o dispersión de un conjunto de valores. Es útil para analizar la dispersión de los datos numéricos. Cuanto mayor sea la desviación estándar, mayor será la variación de los datos.

```sql
SELECT STDDEV(columna) FROM table;
```

MEDIAN() (Mediana): Devuelve la mediana de los valores, es decir, el valor medio cuando los valores están ordenados. La mediana es útil para encontrar un valor representativo en un conjunto de datos que puede tener valores extremos o asimetría. No todos los sistemas de bases de datos soportan esta función nativamente.

```sql
SELECT MEDIAN(columna) FROM table;
```

Uso de funciones de agrupación con GROUP BY

La cláusula **GROUP BY** en SQL es una herramienta fundamental para agrupar filas en subconjuntos que comparten valores comunes en una o más columnas. Esto permite realizar cálculos agregados, como sumas, promedios o conteos, sobre cada uno de estos grupos. Su implementación es muy variada, nos permite contestar preguntas con base a los grupos que involucren consultas complejas, funciones de agrupación e incluso filtrado de datos mediante cláusulas como **HAVING**, **LIKE**, y **WHERE**. Para poder implementar una agrupación con funciones de agrupación, la sintaxis básica a usar es:

```sql
SELECT columna1, columna2, ... , función_agregada(columna)
FROM table
GROUP BY columna1, columna2, ...;
```

La cláusula **HAVING** se utiliza para filtrar grupos de datos después de aplicar GROUP BY y las funciones de agregación. Funciona de manera similar a WHERE, pero mientras WHERE filtra filas **antes de la agrupación**, HAVING filtra grupos **después** de la agregación.

```
SELECT columna1, función_agregada(columna)
FROM tabla
GROUP BY columna1
HAVING condición;
```

La cláusula **LIKE** se utiliza para buscar coincidencias de patrones en cadenas de texto. Puede usarse junto con GROUP BY en la cláusula WHERE o en HAVING.

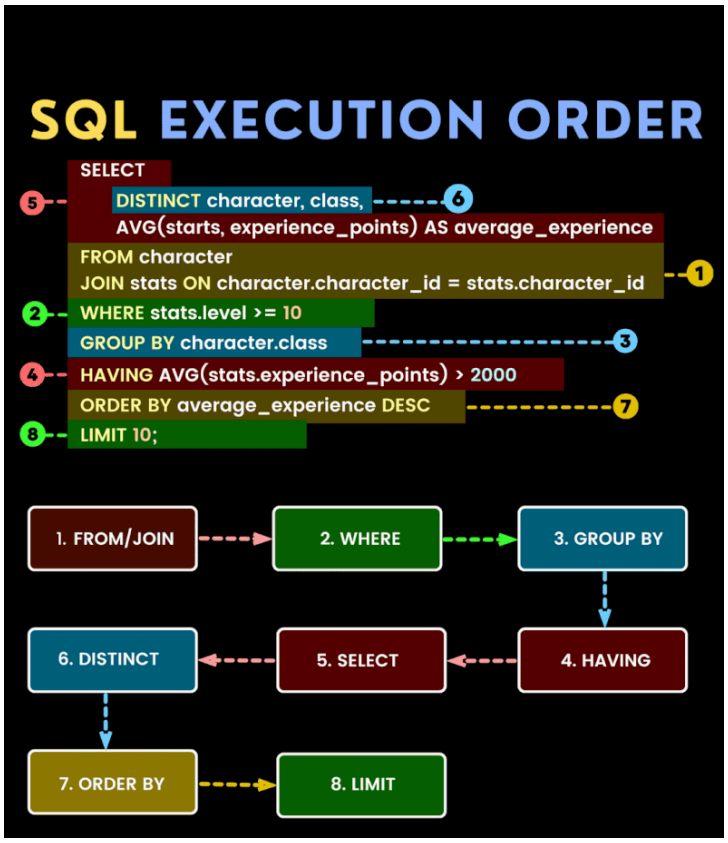
```
SELECT categoria, funcion_agregada(*)
FROM table
WHERE categoria LIKE 'expresion_regular'
GROUP BY categoria;
```

La clausula LIKE puede seguir múltiples patrones de búsqueda según sea el interés del usuario, unos de ellos son:

- **'A%':** Busca todos los elementos que empiecen por A.
- **'%A':** Busca todos los elementos que terminen por A.
- **'%A%':** Busca todos los elementos que contienen A en alguna parte del texto.
- **'A_':** Busca todos los elementos que contienen A y están seguidos de cualquier otro carácter.
- **'_ _ _ _ A':** Busca todos los elementos que tengan exactamente el número de objetos especificados por el comodín _ pero que en su último elemento, terminen por A.
- **'A_ _ _ _ _':** Busca todos los elementos que tengan exactamente el número de objetos especificados por el comodín _ pero que en su primer elemento sea A.
- **'A_ _ _ _ _B':** Busca todos los elementos que contengan la estructura especificada, donde el comodín _ especifica caracteres no conocidos por el usuario que pueden ser cualquiera a los explícitamente indicados en la expresión de búsqueda.

Consultas esenciales en SQL mediante DML

Una vez hemos definido nuestras herramientas básicas para realizar consultas y hemos definido una base de datos relacional básica, para poder ejecutar correctamente consultas a nuestra base de datos debemos tener en cuenta que cada cláusula de la consulta sigue un orden jerárquico representado por el orden en la que cada cláusula es interpretada y ejecutada por nuestro motor de base de datos SQL. Con esto en mente, toda consulta tendrá el siguiente orden de ejecución.



Conocer el orden de ejecución de las cláusulas elementales de SQL es fundamental para entender la apropiada estructuración de un query. Por otra parte, existen unas cláusulas básicas que complementan aquellas que hasta ahora hemos conocido:

- **CAST:** Se utiliza para convertir un valor de un tipo de dato a otro. Por ejemplo, si tienes un número en formato de texto y quieres usarlo como un número entero, usarías CAST(data_text AS NUMERIC) para ejecutar una transformación de tipo correcta.

```
SELECT CAST('123' AS INTEGER)
```

- **WITH:** La sentencia WITH permite crear una consulta temporal, conocida como una CTE (Common Table Expression), la cual es empleada en una consulta posterior. Es útil para simplificar consultas complejas o anidadas.

```
WITH temp AS (
    SELECT id, name FROM patients
)
SELECT * FROM temp;
```

- **CASE:** Permite realizar decisiones en las consultas SQL, similar a un "if" en otros lenguajes. Puede ser usada para devolver diferentes resultados según condiciones. Para emplear CASE, es necesario especificar los casos condicionales mediante la cláusula WHEN y especificar el resultado de cada condición mediante THEN siendo el apartado ELSE el lugar en donde se especifica el resultado alternativo a toda condición en caso de que las condiciones previas no se cumplan. Por otra parte, para poder cerrar la cláusula condicional, empleamos END.

```sql
SELECT name,
    CASE
        WHEN age < 18 THEN 'Minor'
        ELSE 'Adult'
    END AS age_group
FROM patients;
```

- **BETWEEN:** La cláusula BETWEEN en SQL se utiliza para filtrar registros que se encuentran dentro de un rango de valores. Funciona con tipos de datos numéricos, fechas y cadenas.

```sql
SELECT *
FROM patients
WHERE age BETWEEN 30 AND 40;
```

- **IN:** La cláusula IN en SQL se utiliza para filtrar registros que coinciden con alguno de los valores especificados en una lista. Simplifica las consultas que requieren múltiples comparaciones usando el operador OR.

```sql
SELECT column_name
FROM table_name
WHERE column_name IN (value1, value2, value3, ...);
```

Cuando necesitamos generar una tabla muy específica no presente en el esquema relacional sobre la cual podamos generar otra consulta que responda a nuestra pregunta de interés, las **consultas anidadas**, también conocidas como subconsultas, son consultas SQL que se encuentran dentro de otra consulta. Se utilizan para realizar operaciones complejas y permiten obtener resultados que dependen de los resultados de una consulta interna. Una consulta anidada se puede incluir en varias cláusulas SQL, como SELECT, FROM, WHERE, o HAVING.

Los tipos de consultas anidadas son:

- **Correlacionadas:** Dependen de la consulta externa y se evalúan para cada fila de la consulta externa.

```sql
SELECT p.name
FROM patients p
WHERE EXISTS (SELECT p.name
              FROM appointments a
              WHERE a.patient_id = p.id AND a.appointment_datetime > '2023-01-01');
```

- **No correlacionadas:** Se pueden ejecutar de forma independiente y su resultado no depende de la consulta externa.

```sql
SELECT name
FROM patients
WHERE id IN (SELECT patient_id
             FROM appointments
             WHERE appointment_datetime > '2023-01-01');
```

Con estos conocimientos en mente, los tipos de consultas más importantes dentro de SQL son:

- **Consulta de elementos únicos en un campo:** El siguiente ejemplo ilustra como detectar los nombres únicos de una tabla de pacientes,

```
SELECT DISTINCT name
FROM  patients;
```

- **Selección de campos de una tabla con y sin condición de filtrado:** El siguiente ejemplo escoge unas columnas provenientes de una tabla si y solo si se cumple una condición con respecto a un campo.

```
SELECT name, id, gender, insurance_id, email
FROM patients
WHERE date_of_birth > '22-04-2024';
```

- **Consulta de registros bajo un patrón de texto:** El siguiente ejemplo selecciona los registros que cumplen un patrón de texto en un campo especificado.

```
SELECT p.id, p.name, p.date_of_birth
FROM patients p
WHERE p.name LIKE 'A%'
```

- **Consulta bajo múltiples condiciones:** El siguiente ejemplo selecciona aquellos registros que cumplen simultáneamente varias condiciones.

```
SELECT treatment_id, medication_name, dosage
FROM prescriptions
WHERE (dosage > 1 and dosage < 3) and (frequency > 50);
```

- **Selección de registros de acuerdo a un intervalo especificado:** El siguiente ejemplo, selecciona aquellos registros que se encuentran entre dos fechas.

```
SELECT *
FROM appointments
WHERE appointment_datetime BETWEEN '2024-01-01' AND '2024-12-31';
```

- **Ordenamiento de la tabla con respecto a un campo:** El siguiente ejemplo, ordena los registros de una tabla de manera ascendente (ASC) o descendente (DESC) según un campo especificado.

```
SELECT *
FROM treatments
ORDER BY duration DESC;
```

- **Agrupamiento de una tabla con respecto a un campo:** El siguiente ejemplo, agrupa los datos de una tabla con respecto a una columna, aplicando posteriormente una función de agrupación que resume la información contenida en cada grupo.

```
SELECT gender, COUNT(*) as cuenta_genero
FROM patients
GROUP BY gender
ORDER BY cuenta_genero DESC;
```

- **Agrupamiento de una tabla y filtrado por grupos:** El siguiente ejemplo, agrupa los datos de una tabla aplicando un filtro previo a la agrupación, y un filtro posterior a la

agrupación donde el filtro seleccionará aquellos registros dentro de cada grupo que cumple con el criterio de selección.

```sql
SELECT medication_name, ROUND( AVG(CAST(dosage AS NUMERIC)) ,3) AS dosage_promedio
FROM prescriptions
WHERE (CAST(dosage AS NUMERIC)) > 1
GROUP BY medication_name
HAVING AVG((CAST(dosage AS NUMERIC))) > 3
ORDER BY dosage_promedio DESC;
```

- **Consulta por uniones:** Los siguientes ejemplos muestran una consulta resultante a partir de una unión izquierda, derecha, interna y externa respectivamente.

```sql
SELECT p.name, p.gender, p.email, i.name, i.contact_number
FROM patients AS p
LEFT JOIN insurance_companies AS i ON p.insurance_id = i.id;
```

```sql
SELECT p.name, p.specialty, h.name AS hospital_name, h.contact_number
FROM physicians AS p
RIGHT JOIN hospitals AS h ON p.hospital_id = h.id;
```

```sql
SELECT t.appointment_id, t.description, t.duration, p.medication_name, p.dosage, p.frequency
FROM treatments AS t
JOIN prescriptions AS p ON t.id = p.treatment_id
ORDER BY  p.dosage DESC;
```

```sql
SELECT t.appointment_id, t.description, t.duration, p.medication_name, p.dosage, p.frequency
FROM treatments AS t
FULL OUTER JOIN prescriptions AS p ON t.id = p.treatment_id
ORDER BY  p.dosage DESC;
```

- **Consulta por unión y agrupación:** El siguiente ejemplo genera una unión entre tablas agrupando el resultado final con base a un campo especificado.

```sql
SELECT p.id AS patient_id, p.name,  COUNT(a.id) AS appointment_count
FROM patients p
LEFT JOIN appointments a ON p.id = a.patient_id
GROUP BY p.id, p.name
ORDER BY appointment_count DESC;
```

- **Consulta de múltiples joins:** El siguiente ejemplo, muestra una consulta que genera una tabla producto de la unión sucesiva de múltiples tablas a través de campos comunes, la consulta se genera como la consecutiva aplicación de uniones. Posteriormente, se emplea una agrupación y conteo de elementos por grupo, para finalmente organizar los datos con base al conteo de lo elementos en cada grupo.

```sql
SELECT h.name AS hospital_name, COUNT(DISTINCT p.id) AS number_of_patients
FROM hospitals h
JOIN physicians ph ON h.id = ph.hospital_id
JOIN appointments a ON ph.id = a.physician_id
JOIN patients p ON a.patient_id = p.id
GROUP BY h.name
ORDER BY number_of_patients DESC;
```

- **Consultas anidadas no correlacionadas:** El siguiente ejemplo ilustra una consulta anidada no correlacionada donde se emplea la subconsulta como criterio de selección para ejecutar el filtro. Por otra parte, el segundo ejemplo emplea una consulta anidada para usar la tabla interna como los datos de consulta para finalmente calcular un promedio general.

```sql
SELECT name
FROM patients
WHERE id IN (SELECT patient_id
             FROM appointments
             WHERE appointment_datetime > '2024-01-01');
```

```sql
WITH patient_counts AS (
    SELECT ph.id AS physician_id, COUNT(DISTINCT a.patient_id) AS number_of_patients
    FROM physicians ph
    JOIN appointments a ON ph.id = a.physician_id
    GROUP BY ph.id
)
SELECT AVG(number_of_patients) AS average_patients_per_physician
FROM patient_counts;
```

- **Consultas correlacionadas:** El siguiente ejemplo emplea una subconsulta correlacionada con la consulta externa para determinar un criterio de filtro que le permita seleccionar elementos específicos de la tabla externa.

```sql
SELECT p.name
FROM patients p
WHERE EXISTS (SELECT p.name
              FROM appointments a
              WHERE a.patient_id = p.id AND a.appointment_datetime > '2023-01-01');
```


```sql
SELECT p.name
FROM patients p
WHERE EXISTS (SELECT p.name
              FROM appointments a
              WHERE a.patient_id = p.id AND a.appointment_datetime > '2023-01-01');
```


Comentarios Finales

A lo largo de este breve curso de fundamentos de SQL, hemos recorrido un camino extenso, comenzando con las bases fundamentales de bases de datos hasta llegar a temas más avanzados. El objetivo del presente curso introductorio ha sido proporcionar una comprensión clara y sólida de los conceptos esenciales, al mismo tiempo que nos hemos adentrado en herramientas más complejas para la gestión y manipulación de bases de datos. No obstante ya que SQL es un tema tan extenso, te invitamos a continuar tu camino de aprendizaje en bases de datos mediante programas complementarios a nuestro diplomado que te permitan desarrollar habilidades avanzadas en el manejo de bases de datos. No obstante con los conceptos aquí vistos tendrás una comprensión básica del tema que te permitirán tener una comprensión suficiente para abordar temáticas más profundas.

En este pequeño viaje en el mundo de las bases de datos, has aprendido a cerca de:

1. **Fundamentos de bases de datos.**
2. **Introducción a SQL:**

- Conceptos básicos sobre bases de datos relacionales.
- Reglas de normalización y creación de diagramas ER.
- Uso de sentencias DDL tales como ALTER, CREATE, TABLE, COLUMN y DATABASE.
- Sentencias esenciales DML como SELECT, INSERT, UPDATE, y DELETE.
- Estructura de una consulta y el funcionamiento básico de una base de datos relacional.

3. **Filtrado y ordenamiento de datos:**

- Uso de las cláusulas WHERE, ORDER BY, LIMIT, y cómo emplearlas para obtener resultados más específicos.
- Filtrado avanzado con operadores como LIKE, BETWEEN, e IN.

4. **Funciones de Agregación y Agrupación:**

- Aprendimos a utilizar funciones como COUNT, SUM, AVG, MIN, MAX, y cómo aplicarlas con GROUP BY para obtener información agregada de nuestros datos.
- Filtrado de datos entre grupos mediante la cláusula HAVING y cómo emplearla.

5. **Uniones (Joins):**

- Exploramos en detalle los diferentes tipos de joins tales como JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, y su utilidad para combinar datos de diferentes tablas**.**

6. **Consultas Anidadas:**

- Diferenciamos entre subconsultas correlacionadas y no correlacionadas, comprendiendo cómo estas permiten realizar consultas más avanzadas y eficaces.

7. **Filtos avanzados**

- Uso de funciones de conversión de datos como CAST, que nos permiten transformar el tipo de datos en una consulta.
- Uso de CASE para implementar lógica condicional dentro de las consultas, y WITH para organizar subconsultas de manera más clara y reutilizable.

A pesar de que hemos intentado cubrir una gran cantidad de temas, el uso de SQL es una temática muy extensa para cubrirla en el presente curso introductorio. Así pues, te proponemos una lista a cerca de los temas que podrías explorar en capacitaciones futuras:

1. **Optimización de Consultas:** Uso de consultas mediante **EXPLAIN** y otras herramientas de optimización. Planificación de consultas que contienen conocimiento a cerca de los tiempos de procesamiento de las mismas. Uso de consultas distribuidas.
2. **Gestión de Transacciones:** Uso de transacciones SQL entendidas como una consecución específica de consultas empaquetadas en una misma solicitud. Involucra el uso de cláusulas tales como **BEGIN**, **COMMIT** y **ROLLBACK**.
3. **Uso de Vistas y Procedimientos Almacenados:** Las vistas son entendidas como consultas no temporales creadas sobre la base de datos que permiten consultar tablas de datos específicos sin aumentar el tamaño de la base de datos, son útiles para simplificar consultas o reutilizarlas. Por otra parte los procedimientos almacenados o **TRIGGERS**, son protocolos que son activados como funciones de transformación o consultas hechas sobre la base de datos de manera automática en la presencia de una manipulación de la base de datos que activa el procedimiento, especificada por el administrador de la base de datos. Estas son útiles para automatizar tareas comunes dentro de la bases de datos.
4. **Seguridad y Control de Acceso:** En el manejo de bases de datos SQL, es posible asignar roles y permisos ligados a estos roles de manera que un usuario de la base de datos pueda manipular los datos almacenados o solo pueda leerlos. Involucra el uso de

sentencias tales como **GRANT** y **REVOKE** para gestionar dichos permisos a cada rol creado.

El camino hacia la completa maestría en SQL implica un enfoque continuo hacia el perfeccionamiento, tanto en la eficiencia de las consultas como en la capacidad de gestionar bases de datos a gran escala. Ahora, con las herramientas que tienes a tu disposición, estás listo para afrontar nuevos desafíos y convertirte en un verdadero experto en SQL.